FlexaScale: Reinforcement Learning-Based Intelligent Auto-Scaling Framework for Distributed Microservices on Kubernetes

Project report submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Atmakrishna K Raghavan AM.SC.U4CSE23015

Dattanand U D AM.SC.U4CSE23023

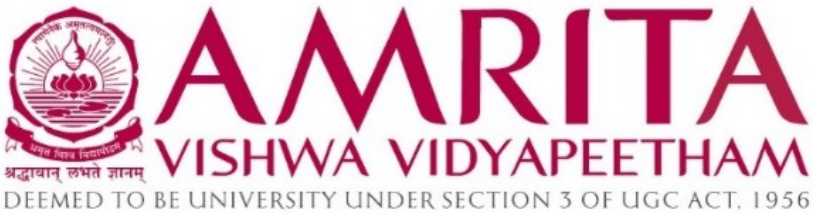
G O Ashwin Praveen AM.SC.U4CSE23027

Ghanshyam S Sunil AM.SC.U4CSE23029

Under the guidance of

Dr. Lakshmi Mohan

Department of Computer Science and Engineering



AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM

AMRITAPURI CAMPUS

KOLLAM 690525

NOVEMBER 2026

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Amrita School of Computing, Amritapuri Campus

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “FlexaScale: Reinforcement Learning-Based Intelligent Auto-Scaling Framework for Distributed Microservices on Kubernetes”, submitted by Atmakrishna K Raghavan (AM.SC.U4CSE23015), Dattanand U D (AM.SC.U4CSE23023), G O Ashwin Praveen (AM.SC.U4CSE23027) and Ghanshyam S Sunil (AM.SC.U4CSE23029), in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Amrita Vishwa Vidyapeetham, is a bonafide record of the work carried out by them under my guidance and supervision at Amrita School of Computing, Amritapuri, during Semester 7 of the academic year 2026 - 2027.

Dr. Lakshmi Mohan
Project Guide
Signature _____

_____
Project Coordinator
Signature _____

_____
Chairperson, CSE
Signature _____

Reviewers
Signature _____

Place: Amritapuri
Date: _____

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Amrita School of Computing, Amritapuri Campus

DECLARATION

We, Atmakrishna K Raghavan (AM.SC.U4CSE23015), Dattanand U D (AM.SC.U4CSE23023), G O Ashwin Praveen (AM.SC.U4CSE23027) and Ghanshyam S Sunil (AM.SC.U4CSE23029), hereby declare that the project entitled “FlexaScale: Reinforcement Learning-Based Intelligent Auto-Scaling Framework for Distributed Microservices on Kubernetes” is a record of our original work carried out under the guidance of Dr. Lakshmi Mohan, Department of Computer Science and Engineering, Amrita Vishwa Vidyapeetham, Amritapuri. To the best of our knowledge, this work has not formed the basis for the award of any degree, diploma, fellowship or similar qualification to any candidate at any university. All sources and contributions of others have been appropriately acknowledged.

Place: Amritapuri

Date: _____

Student details	**Signature of the student**
Atmakrishna K Raghavan AM.SC.U4CSE23015	_____
Dattanand U D AM.SC.U4CSE23023	_____
G O Ashwin Praveen AM.SC.U4CSE23027	_____
Ghanshyam S Sunil AM.SC.U4CSE23029	_____

Project Guide: Dr. Lakshmi Mohan

Signature of the Project Guide: _____

ACKNOWLEDGEMENT

We are deeply grateful to our project guide, Dr. Lakshmi Mohan, Department of Computer Science and Engineering, for her guidance throughout this work. Her review of our literature survey and system design in the earlier phases helped us turn a broad idea into a concrete, testable framework, and her insistence on evidence shaped the way we evaluated it.

We thank the Project Coordinator and the review panel of the Department of Computer Science and Engineering for their feedback during the project reviews, and the Chairperson of the department for providing the academic environment and facilities that made this project possible.

We also thank Amrita School of Computing, Amritapuri, and Amrita Vishwa Vidyapeetham for the opportunity to undertake this project as part of our curriculum. The open-source communities behind Kubernetes, Prometheus, PyTorch, Stable-Baselines3, FastAPI, React and Locust, and Alibaba for publishing its cluster traces, made it possible to build and evaluate a realistic system on student hardware.

Finally, we thank our families and friends for their patience and encouragement during the many long evenings this project required.

ABSTRACT

Applications built from microservices are usually autoscaled on Kubernetes by the Horizontal Pod Autoscaler (HPA), which scales each service on its own CPU utilisation against a fixed target. This rule ignores how services call one another, reacts only after load has arrived and, with its default five-minute scale-down window, keeps spike capacity long after it is needed, wasting compute or degrading latency.

This project designs, implements and evaluates FlexaScale, an autoscaling framework that decides the replica count of every service from a learned policy. Every ten seconds an operator reads telemetry from Prometheus, builds ten unit-free features per service, places them on the application's call graph and runs a graph neural network with a Proximal Policy Optimization (PPO) policy. A safety layer bounds every change, a Kubernetes Lease and an HPA freeze make ownership exclusive, and an Auto mode chooses, per service, whether FlexaScale or the native HPA should be in control. The policy is trained in a simulator fitted to a 16-service e-commerce shop running on Minikube.

End-to-end verification on the live cluster showed Auto mode selecting FlexaScale and applying scaling, mixed ownership under SLA risk, and correct hand-back to HPA when Prometheus was cut or the operator was killed. In a pre-registered evaluation of 46 valid runs, FlexaScale used 7.3% (RL) and 7.7% (Auto) fewer replica-hours than the default HPA, with 95% confidence intervals below zero, but it was on par with a carefully tuned HPA and its latency benefit was not established.

The work relates to SDG 9 (Industry, Innovation and Infrastructure), target 9.4, by making cloud infrastructure more resource-efficient, and to SDG 12 (Responsible Consumption and Production), target 12.2. The measured saving is a demonstrated contribution on a laptop-scale cluster; energy and cost benefits at production scale are expected, not demonstrated.

Keywords: Kubernetes autoscaling; reinforcement learning; graph neural networks; microservices; Proximal Policy Optimization

Table of Contents

Acknowledgement i

Abstract ii

List of Figures iv

List of Tables v

1 Introduction 1

1.1 Background and motivation 1

1.2 Objectives 2

1.3 Scope and constraints 3

1.4 Project report organization 3

2 Problem Definition 4

2.1 Problem statement 4

2.2 Existing system and limitations 5

3 Literature Review 7

3.1 Review scope and selection 7

3.2 Descriptive review of literature works 7

3.3 Comparison of reviewed literature 14

4 Requirements 16

4.1 Functional requirements 16

4.2 Non-functional requirements 17

4.3 Hardware requirements 17

4.4 Software and data requirements 18

5 Proposed Methodology 20

5.1 Design (architecture) or conceptual framework 20

5.2 Proposed approach and procedure 25

6 Results and Analysis 31

6.1 Evaluation setup 31

6.2 Evaluation criteria 32

6.3 Result 33

6.4 Comparative Analysis 41

6.5 Discussion and Interpretation 42

6.6 Limitations 43

7 Conclusion 44

7.1 Conclusion 44

7.2 Future work 44

References 46

A Source Code and Reproducibility 48

A.1 Setup and execution 48

A.2 Supporting material 49

List of Figures

5.1 FlexaScale system architecture....20
5.2 Call graph of the 16-service shop used by the policy....21
5.3 Graph policy network: per-service embedding, two rounds of message passing, focal readout and PPO heads....22
5.4 Auto mode decision rules, applied per service every 10 s....23
5.5 Ownership sequence: Lease, HPA freeze, scaling and hand-back....24
5.6 Twin fidelity against live calibration runs on the second laptop (p95 latency)....26
5.7 Training pipeline of the v7 and v9 policies....28
5.8 Validation reward of the v9 training runs and the teacher rule....28
5.9 The live control loop....29
6.1 Total ready replicas in four verification runs....35
6.2 Dashboard replay of the operator-kill run: the fallback marker and HPA scaling afterwards....35
6.3 v9 evaluation: replica-hours, median and tail latency and SLO-violation time per arm..37
6.4 Replica-hours: paired differences with 95% confidence intervals....37
6.5 End-to-end p95 latency: paired differences with 95% confidence intervals....38
6.6 E1 flash sale: total ready replicas and end-to-end p95 under default HPA, tuned HPA and Auto....39
6.7 Live lab during an Auto-mode flash-sale run: tiles, live autoscaling timeline and per-service replicas....40
6.8 Auto mode page: replay of the SLA-risk run and the four rules with their usage....40
6.9 FlexaScale Auto minus default HPA, by scenario....41

List of Tables

1.1 Sustainable Development Goals addressed by the project....2
1.2 Project objectives, evaluation methods and deliverables....2
3.1 Comparison of reviewed literature....14
4.1 Functional requirements and acceptance criteria....16
4.2 Non-functional requirements with targets and measured values....17
4.3 Hardware used for development, training and evaluation....18
4.4 Software stack and versions....18
5.1 The ten per-service features (observation version 3)....21
5.2 Safety guards applied to every proposal....24
5.3 Differences between the Phase 2 design and the implemented system....25
5.4 Training hyperparameters (v7 and v9 protocol)....27
5.5 Policy versions and what each one taught....30
6.1 Workload scenarios....31
6.2 Evaluation measures....32
6.3 Controller-mode verification on the live cluster (flash sale, 300 s)....33
6.4 Per-service evidence, Auto mode with the SLA-risk variant (run 20261031-035531)....34
6.5 Held-out twin test results (episode reward, higher is better; replica-hours; p95 in ms)...36
6.6 v9 evaluation: mean over valid runs per arm (second laptop, 25 users, 300 s)....36
6.7 Pre-registered paired comparisons (mean Δ with 95% CI)....37
6.8 E1: flash sale, v9 block 4 (Auto) and E2: flash sale, v9 block 10 (RL)....38
6.9 FlexaScale compared with the closest related systems....41
A.1 Repository map....49
A.2 Main groups of automated tests (373 tests in total, all passing)....49

Chapter 1

Introduction

1.1 Background and motivation

Most large web applications today are built as collections of small, independently deployed services. An online shop, for example, may run separate services for the product catalogue, the shopping cart, payments, inventory, fraud checks and notifications, each talking to the others over the network. This *microservice* style lets teams change and scale parts of the system independently, and Kubernetes has become the usual platform for running such systems [18]. Kubernetes keeps a declared number of identical copies (replicas) of every service running and replaces them when they fail.

How many replicas each service needs changes over the day. A flash sale or a marketing campaign can multiply traffic within seconds, while nights are quiet. Too few replicas make pages slow and requests fail; too many waste compute that is paid for and powered whether it is used or not. Kubernetes addresses this with the Horizontal Pod Autoscaler (HPA), which periodically compares each service's average CPU utilisation with a target and adjusts the replica count proportionally [19]. The HPA is simple, predictable and well understood, which is exactly why it is the default choice in practice.

The same simplicity is also its weakness. The HPA looks at one service and one signal at a time. It does not know that a surge at the API gateway will reach the order and payment services a moment later, and it cannot see a bottleneck that is not CPU, such as a service waiting on a database. It is reactive by design, and its default scale-down stabilisation window of five minutes keeps extra replicas long after a spike has passed. Studies of production microservice traces show that call graphs are deep and highly dynamic [11], and benchmark studies show that a slow service can create back-pressure that degrades the whole application [12]. These observations motivate controllers that look at the application as a whole rather than one service at a time.

Reinforcement learning (RL) is a natural candidate. An RL agent learns a decision rule by trial and error against a reward that can express several goals at once, for example low latency and low cost. Several recent systems apply RL or other learning methods to container autoscaling and report gains over the HPA in their settings [1, 7, 6]. Graph neural networks add a way to reason over the service dependency graph [10, 16]. What is less common is a system that puts such a policy in charge of a real cluster *safely*: with guarantees that a wrong decision cannot do serious harm, that two controllers never fight over the same service, and that the native HPA takes over whenever the learned controller

is unsure or unhealthy. FlexaScale was built to provide exactly that, and then to measure honestly whether the learned policy is better than a well-configured HPA.

1.1.1 Sustainable Development Goal mapping

The project relates to two United Nations Sustainable Development Goals [21]. Table 1.1 lists the goals, the relevant targets and how the project maps to them. The mapping distinguishes what this report demonstrates from what is only expected at a larger scale.

Table 1.1: Sustainable Development Goals addressed by the project

SDG and target	Mapping to the project	Beneficiaries	Demonstrated vs expected
SDG 9 Industry, Innovation and Infrastructure; target 9.4 (upgrade infrastructure for resource-use efficiency) and 9.1 (reliable, resilient infrastructure)	FlexaScale is an installable add-on that makes autoscaling of existing Kubernetes applications more efficient while a safety layer and HPA fallback keep them reliable.	Platform and DevOps engineers; organisations running microservices; their users.	Demonstrated: 7-8% fewer replica-hours than default HPA in a pre-registered study; verified fallback under failures. Expected: similar behaviour on larger clusters.
SDG 12 Responsible Consumption and Production; target 12.2 (efficient use of natural resources)	Fewer idle replicas mean less allocated compute, and therefore less energy and cost for the same work.	Cloud users paying for capacity; data-centre operators.	Demonstrated: lower allocated CPU-hours on a laptop cluster. Expected, not measured: energy savings in a data centre.

1.2 Objectives

The objectives below refine the five objectives stated in the initial project idea submission and add the safety objective that emerged from the design phase. Each objective is linked to the way it is evaluated and to the deliverable that results from it (Table 1.2).

Table 1.2: Project objectives, evaluation methods and deliverables

No.	Objective	Evaluation method	Deliverable
O1	Design an autoscaling framework for Kubernetes microservices that adapts to changing workloads without changes to application code.	Deploy on a 16-service shop and a 5-service bookstore; check discovery and control of both.	FlexaScale operator, policy files, Helm chart
O2	Develop a reinforcement-learning decision engine that uses live cluster metrics and learns from simulated workload patterns.	Twin validation and held-out test sets; live inference on the cluster.	Trained GNN + PPO policies (v1-v9), model cards
O3	Make decisions dependency-aware by reasoning over the service call graph.	Graph versus no-graph ablation on three held-out test sets.	Graph policy (GraphFocalExtractor), call-graph inference
O4	Guarantee safe operation: bounded actions, exclusive ownership and automatic fallback to the native HPA.	Unit tests; live fault injection (Prometheus outage, operator kill); cleanup checks after every run.	Safety layer, Lease, HPA freeze/restore, watchdog, Auto mode

No.	Objective	Evaluation method	Deliverable
O5	Evaluate the framework against the native HPA on latency, resource use, cost and SLO compliance.	Pre-registered paired live evaluation with 95% confidence intervals against default and tuned HPA.	Evaluation reports, results folder, charts
O6	Demonstrate deployment with real-time monitoring and visualisation.	Live runs observed through the dashboard; Helm install, upgrade, rollback and uninstall tests.	Control-center dashboard, Grafana, Helm lifecycle logs

1.3 Scope and constraints

Intended users. The framework is meant for platform engineers who run microservice applications on Kubernetes and want an autoscaler that is more economical than the default HPA without giving up its safety. The dashboard and the experiment lab also make it a teaching and research tool for comparing autoscalers under identical load.

Included functions. Automatic discovery of an application's services and their HPAs; telemetry collection from Prometheus; feature construction and graph-based policy inference; per-service arbitration between FlexaScale and the HPA (Auto mode); a safety layer; execution through the Kubernetes API with an exclusive Lease; an experiment lab that runs repeatable comparisons; a web dashboard; and packaging as a Helm chart.

Exclusions. FlexaScale changes only the number of replicas (horizontal scaling). It does not resize pods (vertical scaling), schedule pods onto nodes, or add nodes to the cluster. The policy is not updated online on the live cluster; it is trained offline and only used for inference live. Cost figures are estimates based on public list prices, not bills.

Assumptions and constraints. All experiments run on single-node Minikube clusters on two student laptops, so the load that can be generated is modest (25 to 40 simulated users) and host activity can disturb measurements. Explicit validity rules (Section 6.1) control the second problem; the first limits how far the results generalise. Each service requests 100 millicores of CPU with a 500-millicore limit, and every HPA allows 1 to 10 replicas.

1.4 Project report organization

Chapter 2 defines the problem and examines the native HPA. Chapter 3 reviews and compares twelve research papers. Chapter 4 lists the requirements and the hardware, software and data used. Chapter 5 presents the architecture of FlexaScale and the procedure used to build, train and operate it. Chapter 6 reports the evaluation setup, criteria, verification and benchmark results, comparison, discussion and limitations. Chapter 7 concludes and outlines future work, and Appendix A explains how to reproduce the work.

Chapter 2

Problem Definition

2.1 Problem statement

Consider an application of N microservices deployed on Kubernetes, connected by a directed call graph G in which an edge from service i to service j means that i calls j. At every decision time t the controller observes telemetry for each service (CPU and memory use, request rate, error rate and latency percentiles) together with the current desired and ready replica counts. It must choose a replica count for every service within its bounds.

The goal is to serve the offered load with as few replica-hours as possible while keeping the end-to-end 95th-percentile latency within a service-level objective (SLO) of 1000 ms and keeping failures low. Three practical requirements make the problem harder than an optimisation exercise:

- **Safety.** A learned controller can be wrong. Its actions must be bounded so that a single bad decision cannot overload the node or remove all capacity from a busy service.
- **Exclusive control.** Kubernetes already runs an HPA for each service. If two controllers set the same replica count, they fight and the result is unpredictable. Exactly one controller must own each service at any time.
- **Graceful fallback.** When telemetry disappears, the controller crashes, or the policy is unsure, the application must still be autoscaled, which in practice means returning control to the native HPA.

Stakeholders. Platform engineers who operate the cluster, application teams whose services are scaled, end users who experience the latency, and the organisation that pays for the allocated capacity.

Inputs. An application policy file naming the namespace, label selector, replica bounds, safety parameters and (optionally) the call graph; live metrics from Prometheus; Deployment and HPA objects from the Kubernetes API; and a trained policy checkpoint.

Outputs. Replica changes applied through the Kubernetes scale sub-resource for services FlexaScale owns, restored HPA bounds for services handed back, a decision log that records every proposal, owner, rule and safety reason, and the telemetry and events shown on the dashboard.

Consequences of not solving it. Over-provisioning wastes money and energy; under-provisioning breaks the SLO and loses users; and an unsafe learned controller is worse than none, because it can take an application down.

2.2 Existing system and limitations

2.2.1 The native Horizontal Pod Autoscaler

The HPA is a control loop inside the Kubernetes controller manager. Every 15 seconds by default it reads the average utilisation u of the scaled pods relative to their CPU request and computes the desired replica count with Equation 2.1, where r is the current replica count and u^* is the target utilisation [19].

$$r_{\text{desired}} = \left\lceil r \cdot \frac{u}{u^*} \right\rceil \tag{2.1}$$

Changes within a tolerance of 10% are ignored, and scale-down recommendations are smoothed over a stabilisation window that defaults to 300 seconds, so the highest recommendation of the last five minutes is used. In the shop studied here, every service has an HPA with bounds 1 to 10 and a target of 70% of its CPU request.

2.2.2 Observed limitations

The limitations of this rule are not only theoretical; the project's own measurements show each of them.

1. **It looks at one service and one signal.** At 40 users on the Fedora laptop, the order service's latency rose about tenfold while its CPU stayed at 30-50% of its limit, because it waits on synchronous database calls. A CPU rule (learned or not) does not see this kind of bottleneck directly.
2. **It keeps spike capacity for too long.** In the flash-sale run E1 (Chapter 6), the default HPA added replicas after the spike had begun, peaked at 27 replicas and kept them for five minutes after the spike ended; the run still spent 30 seconds above the SLO.
3. **One fixed target does not suit every service.** In the checkout-heavy runs E3 and E4, the default HPA scaled three or four services to two replicas for a brief CPU rise that did not need it, which cost 14-15% more replica-hours than holding them at one replica.
4. **A tuned HPA helps, but tuning is manual.** Raising the target to 350% of the request and shortening the window to 60 seconds removed most of the waste (Chapter 6), but the right values differed between the two laptops used in this project and had to be found by a calibration experiment.

2.2.3 Other existing approaches

The Vertical Pod Autoscaler and Google's Autopilot adjust pod resource limits rather than replica counts [8]. Event-driven autoscalers scale on queue lengths or custom metrics, but still apply a threshold rule per service. Research systems use machine learning to localise the critical service [7, 9], predict load [2, 4], or learn a scaling policy with RL [1, 3, 6]. Chapter 3 reviews them in detail. Most report gains against the default HPA in their own

setting; few compare against a *tuned* HPA, few address how a learned controller shares a cluster safely with the native one, and few evaluate under rules fixed before the experiments.

2.2.4 Gap addressed by this project

The gap FlexaScale addresses is therefore twofold. On the engineering side, it is the absence of a practical, application-agnostic framework that lets a learned, dependency-aware policy control a live Kubernetes application with bounded actions, exclusive ownership, per-service arbitration and automatic fallback to the HPA. On the evaluation side, it is the absence of a fair comparison: one that pairs runs under identical load, compares against a tuned HPA as well as the default, and fixes the analysis before the results are seen.

Chapter 3

Literature Review

3.1 Review scope and selection

The review covers autoscaling and resource management for containerised microservices, with emphasis on learning-based methods and on methods that account for dependencies between services. Searches were made in IEEE Xplore, the ACM Digital Library, ScienceDirect, the USENIX proceedings and arXiv, using combinations of the keywords *Kubernetes autoscaling*, *horizontal pod autoscaler*, *reinforcement learning*, *microservices*, *service dependency*, *graph neural network*, *SLO* and *resource management*. The period considered is 2019 to 2025.

A paper was included if it (a) proposes or characterises a resource-management method for containers or microservices, (b) evaluates it on a real cluster, a benchmark application or a public trace, and (c) is a primary source with complete bibliographic details. Twelve papers were selected. The first five were identified in the initial literature survey of this project; the other seven were added during design and implementation because they informed specific decisions: the use of a call graph, the choice of workloads and the evaluation against a tuned baseline. Foundational works on PPO, graph neural networks and Kubernetes are cited where they are used but are not part of the comparison.

3.2 Descriptive review of literature works

Each paper is described in one connected paragraph covering its objective, contribution, approach, study context, evaluation, findings and limitations, and what it means for this project. Numerical results are quoted only where the paper states them; otherwise the findings are described qualitatively.

3.2.1 Gwydion

Santos, Reppas, Wauters, Volckaert and De Turck (2025) in “Gwydion: Efficient Auto-Scaling for Complex Containerized Applications in Kubernetes through Reinforcement Learning” [1] investigate how to autoscale applications whose microservices depend on one another. The work proposes an RL-based autoscaling framework for Kubernetes in which the agent's state includes information about the dependencies between microservices, and in which the reward combines deployment cost with latency so that the agent is rewarded for meeting performance goals with fewer resources. The study is situated on real Kubernetes deployments of two applications with very different structures, a Redis Cluster and the Online Boutique microservice demo, and the authors make their environments available for reuse. Evaluation compares the learned agents with the

Kubernetes HPA on latency and resource usage under generated load. The authors report that dependency-aware RL agents can reduce resource usage and latency relative to the HPA in their settings. The main limitations, as noted in our initial survey, are that the method focuses on horizontal scaling with predefined reward strategies, and that the comparison is against the HPA's default configuration rather than a tuned one. Gwydion is the closest prior work to FlexaScale and confirmed that dependency information belongs in the state; FlexaScale extends the idea with a graph neural network over the call graph, a safety layer, per-service arbitration with the HPA and a pre-registered comparison against a tuned HPA.

3.2.2 Smart HPA and ProSmart HPA

Ahmad, Treude, Wagner and Szabo (2025) in “Towards Resource-Efficient Reactive and Proactive Auto-Scaling for Microservice Architectures” [2] address the resource inefficiency of the standard HPA when microservices compete for limited capacity. The work proposes Smart HPA, a reactive autoscaler that manages resources across microservices, and ProSmart HPA, a proactive variant that adds machine-learning-based workload prediction so that scaling can begin before load arrives. The study runs on a Kubernetes deployment of a microservice benchmark application under generated workloads. The authors evaluate the approaches against the Kubernetes HPA using resource utilisation and the frequency of overutilisation and underprovisioning as criteria, and report that both variants use resources more efficiently than the HPA in their experiments, with the proactive variant reacting earlier. The approach does not use reinforcement learning, so it cannot learn a policy that trades latency against cost continuously, and dependencies between services are not modelled explicitly. For FlexaScale, this work showed that substantial gains are possible from better rules alone, which is why our evaluation includes a tuned HPA as a baseline rather than only the default.

3.2.3 RL-based Kubernetes autoscaling

Pandey (2025) in “Reinforcement Learning-Based Autoscaling for Cost and Performance Optimization in Kubernetes Clusters” [3] investigates whether an RL agent can jointly optimise cost, latency and SLA compliance in Kubernetes. The work proposes a multidimensional autoscaler that controls horizontal and vertical scaling together, implemented with the PPO and DDPG algorithms. The study is situated on a managed Kubernetes cluster (Amazon EKS) running Spark jobs from the TPC-DS benchmark. Evaluation compares the learned autoscaler with conventional autoscaling on cost and performance measures. The author reports that the RL-based approach can lower cost while maintaining performance; exact dataset sizes and run counts are not reported in the version reviewed. The main limitations are the extensive training the agent requires and the focus on cost-latency optimisation without modelling dependencies between services. For FlexaScale, this paper supported the choice of PPO as the learning algorithm and highlighted the training cost, which we addressed by training in a fitted simulator and starting from imitation of a simple rule.

3.2.4 AdaptScale

Shi, Dong, Zhang, Liu, Zheng, Cui and Zhang (2020) in “AdaptScale: An Adaptive Data Scaling Controller for Improving Multiple Performance Requirements in Clouds” [4] study how to satisfy several SLA requirements at once while minimising resource usage in cloud data services. The work proposes a controller that combines adaptive workload prediction with a greedy heuristic that selects scaling actions. The study is conducted on a simulation platform rather than a production system. Evaluation measures SLA satisfaction and resource usage against reference controllers, and the authors report that adaptive prediction helps meet multiple requirements with fewer resources. The approach targets cloud data scaling rather than Kubernetes microservices, and a greedy heuristic cannot learn long-horizon trade-offs. Its relevance to FlexaScale is the idea of treating several performance requirements together in one objective, which our reward function does with explicit weights for replicas, latency inflation, SLO violation, failures and churn.

3.2.5 KubCG

El Haj Ahmed, Gil-Castiñeira and Costa-Montenegro (2021) in “KubCG: A Dynamic Kubernetes Scheduler for Heterogeneous Clusters” [5] investigate scheduling of containerised jobs on clusters that mix CPU and GPU nodes. The work proposes a dynamic scheduler for Kubernetes that uses historical execution information to estimate how long a job will take on each kind of node and places it accordingly. The study is situated on a heterogeneous Kubernetes cluster with CPU and GPU resources running machine-learning workloads. The authors evaluate the scheduler against the default Kubernetes scheduler on job completion time, and report that using execution history improves placement decisions in their experiments. The work addresses where pods run rather than how many replicas a service needs, so it does not solve the runtime autoscaling problem. It informed FlexaScale in a practical way: like KubCG, FlexaScale uses measurements of the actual application (pod start-up times and CPU per request) rather than assumed constants, and it keeps placement decisions with the default scheduler.

3.2.6 Horizontal and vertical scaling with RL

Rossi, Nardelli and Cardellini (2019) in “Horizontal and Vertical Scaling of Container-based Applications using Reinforcement Learning” [6] study elasticity for containerised applications where both the number of containers and the CPU share of each can be changed. The work proposes RL policies, including a model-based approach that exploits partial knowledge of the system dynamics, and compares them with model-free Q-learning and Dyna-Q. The study is situated in simulation and in a prototype built on Docker Swarm, with a cost function that weights performance penalties, resource cost and adaptation cost. The authors report that the model-based approach learns a good policy much faster than the model-free alternatives, which need many interactions before they behave well. The limitations are the small, single-application setting and the absence of service dependencies. For FlexaScale this paper made two points concrete: the reward must charge for adaptation (we include a churn term), and learning directly on a live system is too slow, which led us to train in a simulator fitted to our own application.

3.2.7 FIRM

Qiu, Banerjee, Jha, Kalbarczyk and Iyer (2020) in “FIRM: An Intelligent Fine-grained Resource Management Framework for SLO-Oriented Microservices” [7] address SLO violations in microservice applications caused by resource contention. The work proposes a two-level framework: a support vector machine localises the microservice instances on the critical path that are responsible for an SLO violation, and a reinforcement-learning agent based on deep deterministic policy gradients reprovisions low-level resources for them. The study runs on a Kubernetes cluster with benchmark microservice applications (from DeathStarBench and Train-Ticket) and injected performance anomalies. Evaluation measures SLO violations and requested CPU against Kubernetes autoscaling and other baselines. The authors report up to 16 times fewer SLO violations and up to 62% lower requested CPU limits. The approach requires fine-grained tracing to find the critical path and acts on resource limits rather than replica counts. FIRM showed that knowing *which* service is responsible matters; FlexaScale's per-service (focal) decisions over the call graph follow the same intuition without needing distributed tracing.

3.2.8 Autopilot

Rzadca et al. (2020) in “Autopilot: Workload Autoscaling at Google” [8] describe the system Google uses to set the resources and replica counts of jobs in its Borg clusters automatically. The work combines sliding-window recommenders over historical usage with a machine-learning meta-algorithm that chooses among many simple models, and it covers both vertical and horizontal autoscaling. The study is situated in Google's production fleet. Evaluation is based on production measurements of slack (the gap between the limit and actual usage) and of out-of-memory events. The authors report that jobs managed by Autopilot had a slack of 23%, against 46% for manually managed jobs, and far fewer jobs severely affected by out-of-memory errors. The system relies on long histories and large fleets that are not available to a single application, and it does not model call dependencies. For FlexaScale, Autopilot is evidence that simple, conservative, well-engineered rules can be very effective in production, which is why our Auto mode keeps the native HPA as a first-class alternative rather than replacing it.

3.2.9 Sinan

Zhang, Hua, Zhou, Suh and Delimitrou (2021) in “Sinan: ML-based and QoS-aware Resource Management for Cloud Microservices” [9] study how to allocate resources to interactive microservices so that end-to-end quality of service is met with as little resource as possible. The work proposes a manager that trains a convolutional network and a boosted-trees model to predict the end-to-end latency and the risk of a future QoS violation for candidate allocations, and then picks the cheapest safe allocation. The study uses end-to-end applications from DeathStarBench on a local cluster and on Google Compute Engine. Evaluation compares QoS and resource usage with autoscaling and other learning-based managers, and the authors report that Sinan meets QoS while using fewer resources, and that the model's predictions can be inspected to explain decisions. The approach needs a large data-collection phase on the target application. Sinan's emphasis on explainability influenced FlexaScale's decision log, in which every action records the rule and the safety reasons behind it.

3.2.10 GRAF

Park, Choi, Lee and Han (2021) in “GRAF: A Graph Neural Network based Proactive Resource Allocation Framework for SLO-Oriented Microservices” [10] address the difficulty of allocating resources along a microservice call graph so that the end-to-end latency SLO is met. The work proposes a framework that uses a graph neural network to model how latency propagates through the dependency graph and then optimises the CPU allocation of all services together, proactively, as the front-end load changes. The study runs on a Kubernetes cluster with benchmark microservice applications. Evaluation compares resource usage and SLO satisfaction with existing autoscalers, and the authors report lower total CPU allocation for the same latency target. The approach relies on a latency model trained for each application and on accurate load forecasts. GRAF was the most direct motivation for FlexaScale's graph neural network: we use message passing over the call graph to inform each service's decision, and we test with an ablation whether the graph actually helps in our setting.

3.2.11 Alibaba microservice trace analysis

Luo et al. (2021) in “Characterizing Microservice Dependency and Performance: Alibaba Trace Analysis” [11] characterise how microservices behave in a large production environment. The work releases and analyses a trace of Alibaba's microservice clusters that records call dependencies, request rates, response times and resource usage for more than twenty thousand microservices. The study is situated entirely in production data. The analysis uses statistical characterisation of call-graph topology, dynamics and resource interference rather than a controlled experiment. The authors find that call graphs are large, deep and change frequently, that many services are called through stateless chains, and that interference between co-located services affects response times. The trace does not include the autoscaling decisions themselves, so it cannot be used to evaluate a controller directly. FlexaScale used the same trace family (cluster-trace-microservices-v2021) to build the training data for its first six policy versions, and the paper's findings justified building the call graph from the application rather than assuming a fixed shape.

3.2.12 DeathStarBench

Gan et al. (2019) in “An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud and Edge Systems” [12] address the lack of realistic microservice applications for research. The work contributes DeathStarBench, a suite of end-to-end applications (including a social network, a media service, an e-commerce site, a banking system and a drone-swarm IoT application) built from many interacting services. The study runs these applications on real servers and measures their behaviour from the hardware up to the cluster level. Evaluation is a characterisation of latency, network processing, cache behaviour and the effect of dependencies on tail latency. The authors show that microservices spend a large share of their time in network processing and that dependencies create back-pressure, so a slow back-end service can cause quality-of-service violations far upstream. The benchmarks are large to deploy on a laptop. FlexaScale's 16-service shop was designed in the same spirit, with realistic user journeys and a checkout path that fans out to eight services, at a scale that fits a single-node cluster.

3.3 Comparison of reviewed literature

Table 3.1 compares the twelve reviewed works using the same attributes as the descriptive review. Study context covers the setting and the evidence used; details that a paper does not state are marked “Not reported”.

Table 3.1: Comparison of reviewed literature

Work and citation	Problem or objective	Approach and contribution	Study context	Evaluation criteria	Key findings	Limitations and gap
Gwydion [1] Santos et al., 2025	Autoscale dependent microservices efficiently	RL agent with dependency-aware state; cost and latency reward	Kubernetes; Redis Cluster, Online Boutique	Latency, resource usage vs HPA	Lower resources and latency than default HPA	Horizontal only; predefined rewards; default HPA baseline
Smart HPA [2] Ahmad et al., 2025	Resource-efficient HPA	Reactive Smart HPA; proactive ProSmart HPA with ML prediction	Kubernetes microservice benchmark	Utilisation, over- and under-provisioning	More efficient than HPA; proactive reacts earlier	No RL; dependencies not modelled
RL autoscaling [3] Pandey, 2025	Joint cost, latency and SLA optimisation	PPO and DDPG; horizontal + vertical	Amazon EKS; Spark TPC-DS	Cost, performance	Lower cost at maintained performance	Long training; no dependencies; run counts not reported
AdaptScale [4] Shi et al., 2020	Meet multiple SLAs with few resources	Adaptive prediction + greedy heuristic	Simulation platform	SLA satisfaction, resource use	Adaptive prediction helps meet SLAs	Cloud data scaling, not microservices; greedy
KubCG [5] El Haj Ahmed et al., 2021	Scheduling on CPU-GPU clusters	Dynamic scheduler using execution history	Heterogeneous Kubernetes cluster	Job completion time	Better placement than default scheduler	Placement only; no runtime autoscaling
Rossi et al. [6], 2019	Horizontal + vertical container elasticity	Model-based RL vs Q-learning and Dyna-Q	Simulation; Docker Swarm prototype	Weighted cost: performance, resources, adaptation	Model-based RL learns much faster	Single app; no dependencies
FIRM [7] Qiu et al., 2020	Mitigate SLO violations from contention	SVM critical-path localisation + DDPG reprovisioning	Kubernetes; DeathStarBench, Train-Ticket	SLO violations, requested CPU	Up to 16x fewer violations; up to 62% less requested CPU	Needs tracing; acts on limits, not replicas
Autopilot [8] Rzadca et al., 2020	Automate job resources at scale	Sliding-window recommenders + ML meta-algorithm	Google Borg production	Slack, OOM events	Slack 23% vs 46% manual; fewer OOMs	Needs long history and fleet; no dependencies
Sinan [9] Zhang et al., 2021	QoS-aware allocation for microservices	CNN + boosted trees predict latency and violation risk	DeathStarBench; local cluster and GCE	QoS met, resources	Meets QoS with fewer resources; explainable	Large per-app data collection
GRAF [10] Park et al., 2021	Proactive allocation along the call graph	GNN latency model + joint CPU optimisation	Kubernetes; benchmark microservices	CPU allocation, SLO	Lower CPU for the same SLO	Per-app latency model; relies on forecasts

Work and citation	Problem or objective	Approach and contribution	Study context	Evaluation criteria	Key findings	Limitations and gap
Alibaba trace [11] Luo et al., 2021	Characterise production microservices	Trace release and statistical analysis	Alibaba production, >20,000 services	Topology, dynamics, interference	Deep, dynamic call graphs; interference matters	No controller evaluated
DeathStarBench [12] Gan et al., 2019	Realistic microservice benchmarks	Open-source end-to-end applications	Real servers; cloud and edge	Latency, network, tail effects	Dependencies cause back-pressure and tail latency	Heavy to deploy on small hardware

Summary of the gap. Across the twelve works, three patterns stand out. First, learning-based controllers are almost always compared with the HPA in its default configuration, so it is unclear how much of their gain a tuned threshold rule would also obtain. Second, the question of how a learned controller coexists with the native one, who owns a service, what happens when telemetry fails, is rarely treated as part of the design. Third, dependency information is used in different ways (state features, critical-path localisation, graph neural networks), but its benefit is seldom isolated by an ablation. FlexaScale addresses all three: it compares against default and tuned HPA under pre-registered rules, it makes ownership and fallback explicit, and it measures the contribution of the graph separately.

Chapter 4

Requirements

4.1 Functional requirements

The functional requirements in Table 4.1 follow from the objectives in Section 1.2. Each requirement names the actor, the expected behaviour and the acceptance criterion that was used to check it; the last column records where the check is documented. Every requirement listed was met in the final system.

Table 4.1: Functional requirements and acceptance criteria

ID	Requirement (actor: input → behaviour → output)	Acceptance criterion	Verified by
FR1	Operator: policy file → discover Deployments by label, their HPAs, requests and limits → list of managed services	All 16 shop services found, database and frontend excluded; bookstore (5 services) found without code change	Unit tests; live runs
FR2	Operator: Prometheus → CPU, memory, request rate, errors, p50/p90/p95/p99 per service over a 1-minute window	Missing values stay missing (never invented); quantiles ordered p50 ≤ p90 ≤ p95 ≤ p99	test_workload_metrics.py; live check
FR3	Operator: telemetry → 10 unit-free features per service, identical in training and live	Same FeatureTracker code path in both	Code review; tests
FR4	Policy: features + call graph → one action in {−1, 0, +1, +2, +3} per service with a confidence	Decision logged with mode ppo_graph every step	Decision logs
FR5	Auto arbiter: both proposals → owner per service with rule and reason	Rules applied in order with 2-step dwell to FlexaScale	test_auto_mode.py; verification runs
FR6	Safety layer: proposal → approved count within bounds, step limits, budget, cooldowns	Every decision lists accepted or its safety reasons	test_operator.py; logs
FR7	Executor: approved change → Lease renewed, HPA frozen at min = max, Deployment scaled	Kubernetes reports new desired count; no change without ownership	Verification (exclusivity check)
FR8	Executor: hand-back trigger → original HPA bounds restored from annotations	No frozen HPA and no Lease after every run	cleanup.safe in all runs
FR9	Lab: experiment request → comparable start, Locust load, sampling every 5 s, saved run files	run.json, telemetry.csv, decisions.jsonl and Locust CSVs written	Experiments folder
FR10	Lab watchdog: dead or stalled operator → restore every HPA and log a fallback event	HPAs restored within one sample interval	auto_crash run (+1 s)
FR11	Dashboard: live and replayed runs → timeline, ownership, comparison, model pages	Values shown equal the stored values; missing shown as “—”	Screenshots; manual review

ID	Requirement (actor: input → behaviour → output)	Acceptance criterion	Verified by
FR12	Helm chart: install, upgrade, rollback, uninstall with RBAC limited to scaling	Lifecycle script passes all checks; application data preserved	33/33 checks

4.2 Non-functional requirements

Non-functional requirements describe how well the system must behave. Where a requirement can be measured, the measured value from the final system is given beside the target (Table 4.2).

Table 4.2: Non-functional requirements with targets and measured values

Quality	Requirement and target	Measured in the final system
Responsiveness	A decision for every service at least every 10 s; inference must not delay the loop	10 s control period; one CPU forward pass per service, in milliseconds
Actuation delay	New desired replicas visible to Kubernetes within one control step	Desired after 1.1-6.1 s; ready after 6-16 s (pod start) in verification runs
Safety	No action outside bounds; at most +3/−1 per step; total budget 48 replicas	Enforced by guards; no violation in any logged decision
Exclusivity	Never two controllers acting on one Deployment	No exclusivity violation in any verification run
Reliability	Automatic return to HPA on telemetry loss, crash, Lease loss or stop	Prometheus outage: all 16 services to HPA in the same step; SIGKILL: HPAs restored after 1 s
Data honesty	Missing telemetry shown as missing, never estimated; capped quantiles flagged	“—” for missing values; “≥” for quantiles beyond the largest bucket
Security	RBAC limited to reading metrics objects and scaling; dashboard bound to 127.0.0.1	Helm chart ServiceAccount and Role; local-only lab server
Privacy	Only infrastructure metrics collected; no user data	Metrics are CPU, memory, request counts and latency histograms only
Portability	Works on Linux and Windows hosts; any app via a policy file	Runs on Fedora 43 and Windows 11; shop and bookstore
Maintainability	Modular code with automated tests	373 tests pass (pytest, about 34 s)
Reproducibility	Every result traceable to raw run files; evaluation rules fixed in advance	results/ rebuilt from raw files by a script; POLICY_V7/V9 pre-registrations

4.3 Hardware requirements

No dedicated hardware was required beyond the team's laptops. The design phase anticipated a three-node cloud cluster; in the end every experiment ran on single-node Minikube clusters, which kept costs at zero and gave full control over the environment, at the price of limited load (Section 6.6). Table 4.3 lists the two machines used.

Table 4.3: Hardware used for development, training and evaluation

Item	**Laptop 1 (development, v7, verification)**	**Laptop 2 (v9 training and evaluation)**
Operating system	Fedora Linux 43	Windows 11 with Docker Desktop (WSL2)
Processor	Intel Core i5-1340P, 16 hardware threads	AMD Ryzen 7 6800H
Memory	16 GB	Docker VM 7.9 GB
GPU	None (integrated graphics)	NVIDIA RTX 3050, 4 GB
Kubernetes node	Minikube, Docker driver, containerd, Kubernetes v1.31.0, 9 GiB	Minikube, 4 vCPU, 6 GB
Role	Code, tests, v7 training and replication, controller verification, Helm tests	GPU training of v9, pre-registered v9 evaluation

4.4 Software and data requirements

Table 4.4 lists the main software with the versions used in the final system. The design phase planned PyTorch Geometric for the graph network; the final model implements its two rounds of message passing directly in PyTorch, which removed a heavy dependency without changing the method.

Table 4.4: Software stack and versions

Layer	**Software (version)**	**Purpose**
Language	Python ≥ 3.10 (3.14 in the development environment); TypeScript 5.6.3	Operator, lab, training; dashboard
Learning	PyTorch 2.13.0; Stable-Baselines3 2.9.0; Gymnasium 1.3.0	GNN, PPO, simulation environments
Orchestration	Kubernetes v1.31.0 on Minikube; kubectl; Helm 3.19	Cluster, deployment, packaging
Monitoring	kube-prometheus-stack 92.11.0 (Prometheus, Grafana, cAdvisor)	Metrics collection and inspection
Application	FastAPI services, React frontend, PostgreSQL 16 (shopping-app chart 1.1.0)	System under control
Load testing	Locust 2.46.5	User journeys and workload scenarios
Control center	FastAPI 0.141 lab server; React 18.3.1, Vite 5.4.11, Tailwind CSS 3.4.17	Dashboard and experiment API
FlexaScale release	Helm chart flexascale 0.5.1, operator image 0.6.0 (built locally)	In-cluster installation
Testing	pytest	373 automated tests

4.4.1 Data

Three kinds of data were used. **(1) The Alibaba microservice trace** (cluster-trace-microservices-v2021) [11, 20] provides per-minute CPU, memory, instance counts, request rates and response times of production microservices. About 7.4 GB of raw files were joined per service and minute, replicas were counted as distinct instance identifiers, and CPU and memory were converted to percent. The result, 36,355 rows, was split into training, validation and test sets *by service*, so that test services were never seen in training. A unit error in an early version of this pipeline (fractions stored where percent was expected) was found and fixed, and the correction is recorded in a provenance file. This data drove the simulators of policy versions v1 to v6.

(2) Calibration runs of our own shop on each laptop: native-HPA runs at four CPU targets, 25 users and 300 seconds, used both to choose the tuned-HPA baseline and to fit the application twin on which v7 and v9 were trained. **(3) Live telemetry** recorded during every experiment: 5-second samples of replicas, metrics and load, the operator's decision log and Locust's statistics. More than 289 live runs were recorded; invalid runs are kept separately with the reason for exclusion.

Chapter 5

Proposed Methodology

5.1 Design (architecture) or conceptual framework

FlexaScale is built as an add-on that is installed beside an existing application and controls it from outside, through the Kubernetes API, without changes to the application's code. Figure 5.1 shows the complete system. The upper panel is the environment: the application, the load generator used in experiments and the monitoring stack. The middle panel is the FlexaScale operator, which runs one control step every ten seconds. The lower panel is the control center through which a user starts experiments and watches decisions as they happen.

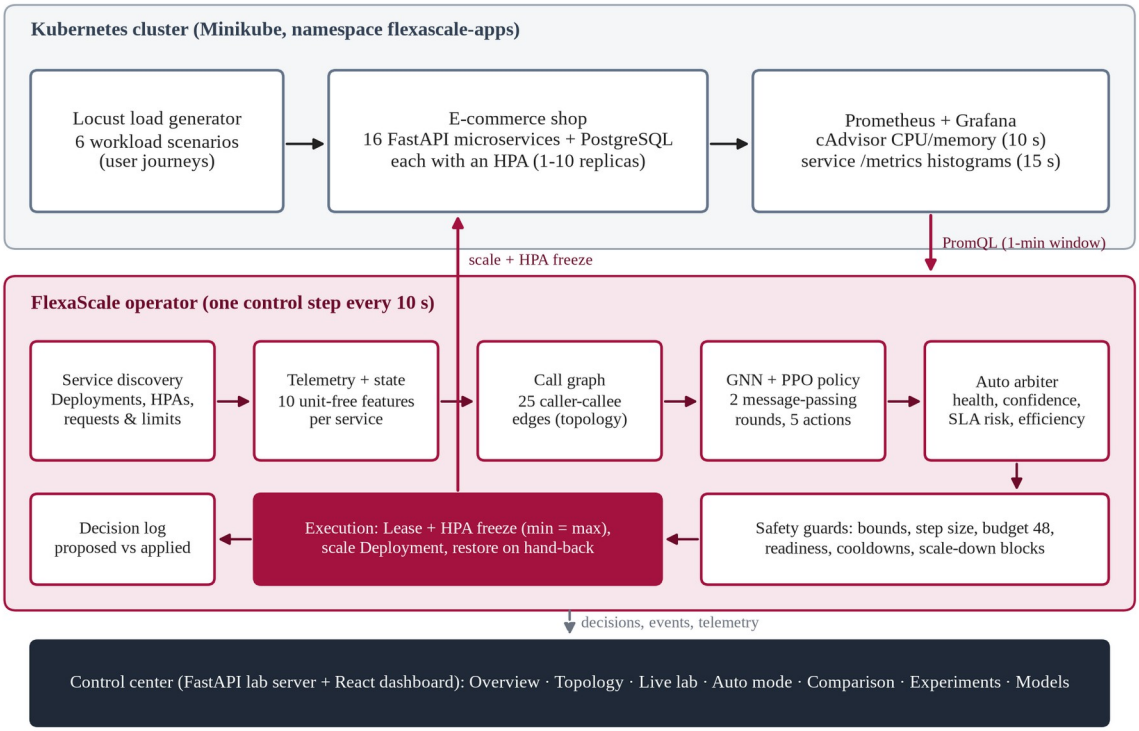


Figure 5.1: FlexaScale system architecture

5.1.1 Application under control

The main test application is an e-commerce shop of 16 Python (FastAPI) microservices with a React frontend and a PostgreSQL database, deployed by a Helm chart. Every service exports a request counter and a latency histogram on a `/metrics` endpoint and has its own HPA. Figure 5.2 shows the call graph used by the policy: the API gateway calls nine services directly, and a checkout in the order service fans out to catalogue, inventory,

fraud, payment, shipping, cart and notification services. A second, five-service bookstore application is used to show that nothing in FlexaScale is specific to the shop.

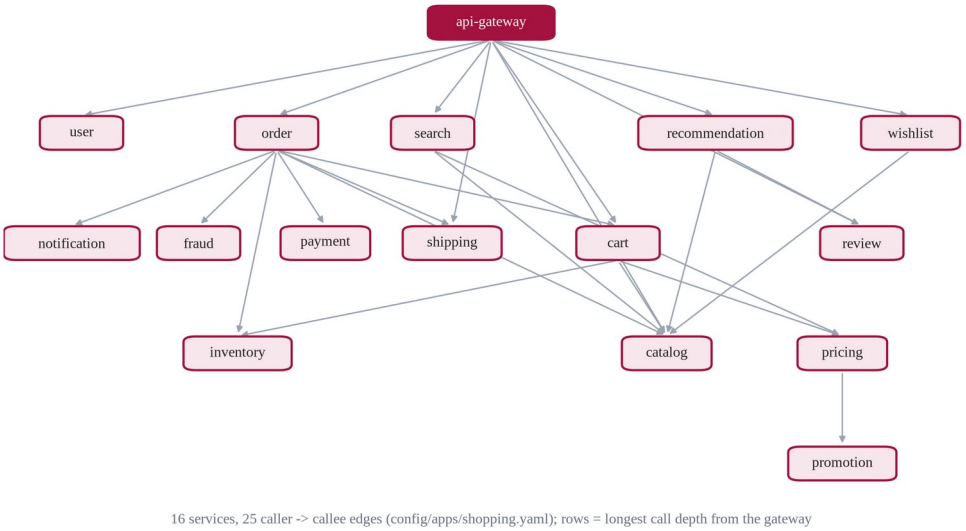


Figure 5.2: Call graph of the 16-service shop used by the policy

5.1.2 Metrics pipeline and service discovery

Prometheus scrapes CPU and memory from cAdvisor every 10 seconds and each service's `/metrics` endpoint every 15 seconds. At each step the operator issues a handful of batched PromQL queries over a one-minute window. Three real-world problems were found and handled explicitly while building this pipeline: cAdvisor on some runtimes reports CPU only per pod, not per container; pods scraped by two jobs had their request rates counted twice; and a service with no requests was being reported as missing rather than as zero. Discovery lists the Deployments carrying the application's label, finds each one's HPA by its scale target rather than by name, and reads requests, limits and replica counts.

5.1.3 State representation

For each service, the operator builds the ten features of Table 5.1. All of them are ratios or logarithms, so that a fast service and a slow service, or a small cluster and a large one, produce comparable numbers. The same code computes the features in the simulator during training and in the operator live, which removes a common source of mismatch between training and deployment.

Table 5.1: The ten per-service features (observation version 3)

#	Feature	Definition	Range
0	cpu_limit_frac	CPU per ready replica divided by the CPU limit	0-2
1	cpu_limit_trend	Change of feature 0 since the previous decision	−1 to 1
2	rps_trend	Request rate now divided by the rate about 60 s earlier	0-3
3	latency_inflation	log2 of latency divided by the service's own low-load latency	−1 to 4

#	Feature	Definition	Range
4	replica_frac	Replicas divided by the maximum (10)	0-1
5	ready_frac	Ready replicas divided by desired replicas	0-1
6	cpu_per_request	log10 of CPU-milliseconds per request, divided by 3	0-1
7	burstiness	log2 of peak over mean request rate in the last 5 minutes	0-3
8	since_scale_up	Seconds since the last scale-up divided by 60 (capped)	0-1
9	since_scale_down	Seconds since the last scale-down divided by 60 (capped)	0-1

The observation given to the network contains these features for up to 20 services, the 20 × 20 adjacency matrix of the call graph, a mask marking which of the 20 slots hold a real service, and a one-hot vector marking the *focal* service whose action is being decided: 640 numbers in total. An application with more than 20 services is refused as incompatible.

5.1.4 Policy network

Figure 5.3 shows the network. Each service's ten features are mapped to a 64-dimensional embedding. Two rounds of message passing then let every service combine its own embedding with the mean embedding of the services it calls and the mean embedding of the services that call it, with separate weights for each direction (Equation 5.1) [17, 16]. The focal service's final embedding is concatenated with the mean embedding of the whole application, and this 128-dimensional vector feeds the PPO actor, which outputs a probability for each of the five replica changes, and, during training only, a critic that estimates the value of the state. The network has 42,566 parameters.

$$h_i^{(k+1)} = h_i^{(k)} + \mathrm{ReLU}\left(\mathrm{LN}\left(W_0^{(k)} h_i^{(k)} + W_1^{(k)} \bar{h}^{(k)}_{\mathrm{callees}(i)} + W_2^{(k)} \bar{h}^{(k)}_{\mathrm{callers}(i)}\right)\right) \tag{5.1}$$

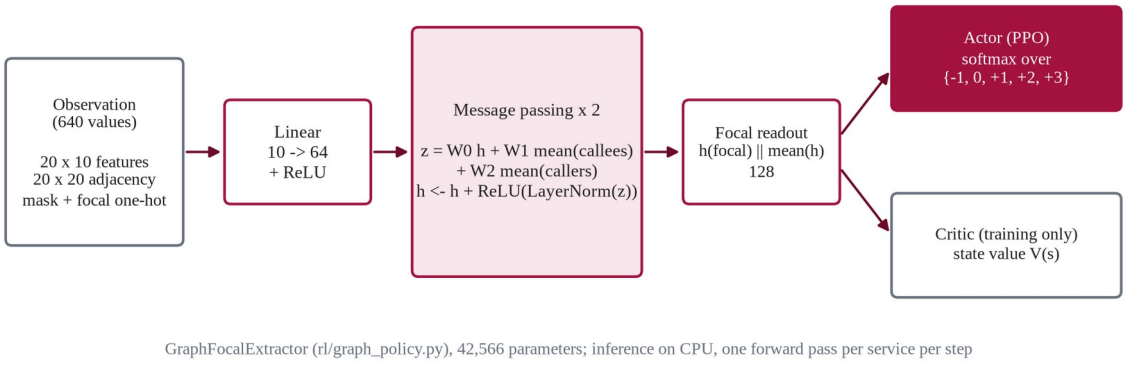


Figure 5.3: Graph policy network: per-service embedding, two rounds of message passing, focal readout and PPO heads

On the live cluster the operator takes the most likely action for each service and reports its probability as the policy's confidence. One forward pass per service per step runs on the CPU in milliseconds.

5.1.5 Auto mode: per-service arbitration

Auto mode decides, for every service and every step, whether FlexaScale or the native HPA should be in charge. Both controllers' proposals are computed from the same telemetry: the policy's action, and the HPA's recommendation from Equation 2.1 at a configured target. Four rules are then checked in order (Figure 5.4); the first that applies decides. Moving a service *to* FlexaScale requires the same choice on two consecutive steps, while moving it back to the HPA for health or SLA risk happens immediately. This asymmetry prevents flapping and makes the safe direction fast.

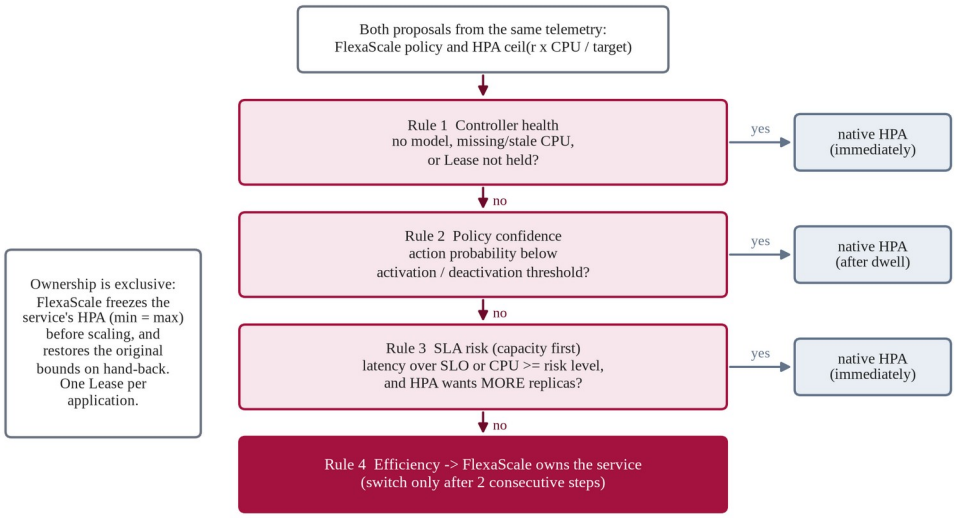


Figure 5.4: Auto mode decision rules, applied per service every 10 s

The SLA-risk rule embodies a *capacity first* principle: when a service is at risk, the controller that wants more replicas wins. In the shop's default policy the CPU risk level is 400% of the request (80% of the limit) and the HPA recommendation is computed at a target of 350%, the tuned value found on the second laptop. The confidence thresholds of rule 2 are set to zero in the shop's policy, so that rule never fires there; Section 5.1.8 explains why.

5.1.6 Safety layer and exclusive ownership

Every proposal passes the guards of Table 5.2 before it can reach the cluster. A guard can reduce or block a change, never enlarge it, and every decision records either `accepted` or the reasons it was changed.

Table 5.2: Safety guards applied to every proposal

Guard	Rule in the shop's policy
Bounds	Never below the service's minimum (1) or above its maximum (10)
Step size	At most +3 replicas up and −1 down per step
Replica budget	At most 48 replicas for the whole application, so the node is not overloaded
Readiness	No scale-up while new pods are still starting; no scale-down while pods are not ready
Cooldowns	20 s after a scale-up and 30 s after a scale-down, so telemetry can catch up
Scale-down blocks	No scale-down while CPU ≥ 400% of request, memory ≥ 90% or latency above the SLO
Telemetry	Missing or stale CPU (older than 120 s): no FlexaScale decision; the service goes to its HPA
Ownership	Act only while holding the application's Lease and after freezing the service's HPA

Ownership is enforced with two Kubernetes mechanisms (Figure 5.5). First, in live mode the operator must hold a `coordination.k8s.io` Lease named after the application; it is renewed every step and expires after 30 seconds, so only one FlexaScale controller can act on an application. Second, before FlexaScale changes a service, it sets that service's HPA to `minReplicas = maxReplicas` equal to the new count and stores the original bounds in annotations on the HPA. A frozen HPA cannot move the replicas, so the two controllers never act on the same Deployment at once. Hand-back restores the original bounds from the annotations. Hand-back happens per service (Auto rule, missing telemetry) or for all services (operator stop, a failed step, a lost Lease, shutdown), and the experiment lab adds a watchdog that restores every HPA itself if the operator process dies or stops updating its status for four poll intervals.

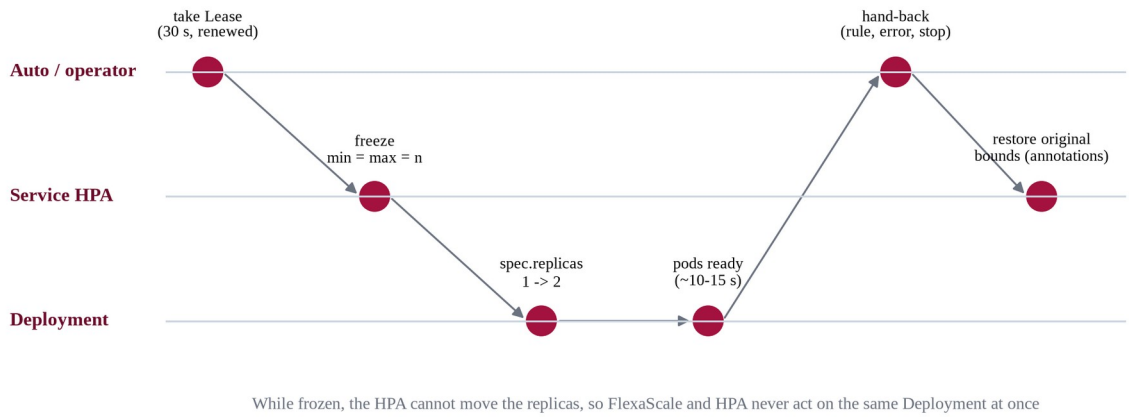


Figure 5.5: Ownership sequence: Lease, HPA freeze, scaling and hand-back

5.1.7 Control center and packaging

The control center is a FastAPI lab server, bound to the local machine only, with a React and TypeScript dashboard. Its seven pages are Overview, Service topology, Live lab, Auto mode, Comparison, Experiments and Models. The Live lab starts a run in one of three modes (native HPA with FlexaScale in shadow, FlexaScale, Auto) and shows a live autoscaling timeline of desired and ready replicas, both controllers' proposals, owner bands, traffic and latency. A single command (`./fsctl start`) brings up the cluster, port-forwards, checks and the dashboard; `./fsctl stop` stops everything and verifies that every HPA has been restored. For installation into a cluster, the operator is packaged as a Helm chart. Its namespaced Role grants only read access in observe mode; the write verbs (patching HPAs and Deployments, updating the scale sub-resource and the Lease) are granted only when the release is installed in live mode. A Helm hook hands every service back to its HPA before an upgrade, a rollback or an uninstall.

5.1.8 Relation to the Phase 2 design

The implementation follows the six-component architecture approved in the design phase. Four points changed during implementation, for the reasons given in Table 5.3.

Table 5.3: Differences between the Phase 2 design and the implemented system

Design element	**What was implemented**	**Reason**
Adaptive reward weighting by traffic phase	A fixed, multi-term reward with documented weights (Equation 5.2)	Weights changing with a detected phase made training targets non-stationary and results hard to attribute; the fixed reward already charges latency inflation, SLO violation, failures, replicas and churn.
Confidence-based delegation to HPA	Implemented as Auto rule 2; thresholds set to 0 in the shop's policy	The policy's action probabilities were not calibrated as a measure of correctness; health and SLA-risk rules gave clearer, testable hand-back conditions.
Online PPO updates on the live cluster	Offline training in a fitted twin; inference only on the cluster	Learning on a live application is slow and risky; a policy that changes during an experiment cannot be evaluated fairly.
Three-node cloud staging cluster	Single-node Minikube on two laptops	Zero cost and full control of the environment; the limitation on load is reported in Section 6.6.

5.2 Proposed approach and procedure

This section explains how the policy is obtained and how the system operates, step by step, in enough detail to reproduce the work. Figure 5.7 summarises the training pipeline and Figure 5.9 the live control loop.

5.2.1 Step 1: Simulation environments

Training on the real cluster would take months and could break the application, so every policy was trained in simulation. Two generations of simulator were used. Versions v1 to

v6 used Gymnasium environments driven by the Alibaba trace, in which each service's load followed a trace segment. Live tests showed that these policies did not transfer well: they had never seen the shop, and the earlier simulators made the HPA's rule close to optimal by construction.

Versions v7 and v9 were therefore trained in an *application twin*, a simulator fitted to measurements of our own shop. The twin models each service's CPU cost per request, its 0.5-core CPU limit and the queueing that follows when the limit is reached, the call graph, pod start-up time (median 10 s, 90th percentile 15 s, measured live), and controllers that see delayed metrics as the real ones do. One step is 10 seconds. Training workloads are random variations of the six live scenarios with 8 to 45 peak users and durations of 300 to 600 seconds, and a quarter of the episodes use a randomly generated application so that the policy must read the graph instead of memorising service names. The v9 twin, fitted to the second laptop, reproduces the calibration runs with a mean absolute error of 2.0% in replica-hours and 8.9% in p95 latency (Figure 5.6).

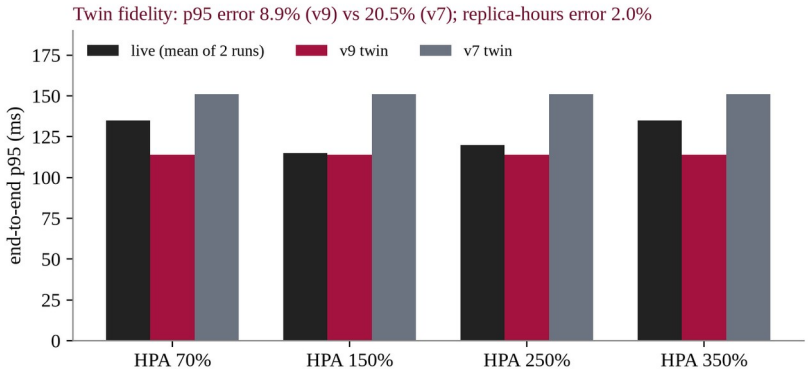


Figure 5.6: Twin fidelity against live calibration runs on the second laptop (p95 latency)

5.2.2 Step 2: Reward function

The reward is computed per 10-second step in units of replica-equivalents, so that each weight answers a concrete question, such as how many extra replicas avoiding an SLO violation is worth (Equation 5.2). The first term counts replicas above each service's minimum; the second charges the inflation of end-to-end p95 latency over its value with ample capacity, capped at four; v is the fraction of the step in which p95 exceeded 1000 ms, f the percentage of failed requests and c the number of replica changes (churn).

$$R_t = -0.1\left(\sum_i (r_i - r_i^{\min}) + 6\min\left(4, \frac{p95_t}{p95^0} - 1\right) + 10\,v_t + 5\,f_t + 0.25\,c_t\right) \tag{5.2}$$

For example, a step with three extra replicas, no slowdown and one change scores −0.325, while a step with no extra replicas but doubled latency and the SLO broken for half the step scores −1.1, so the learner prefers paying for the replicas.

5.2.3 Step 3: Warm start by imitation

Early versions trained with PPO from scratch learned to scale everything up, or failed to match a simple threshold rule that the network could represent exactly. From v5 onward, each run therefore starts by imitating a rule (behaviour cloning: cross-entropy on the rule's actions, with the critic fitted to returns). The teacher for v7 and v9 was chosen from the family `limit_target(t/d)`, which scales up when CPU per replica exceeds a fraction t of the limit and down below d, with $t \in \{0.4, 0.5, 0.6, 0.7\}$ and $d \in \{0.2, 0.3\}$; the validation-best rule was `limit_target(0.6/0.3)`.

5.2.4 Step 4: PPO fine-tuning

PPO [13] improves the policy by maximising the clipped surrogate objective of Equation 5.3, where ρ is the ratio of the new to the old probability of the action taken and Â is the advantage estimated with generalised advantage estimation (Equation 5.4) [14]. The clip keeps each update close to the previous policy, which keeps training stable. The implementation uses Stable-Baselines3 [15] with the settings in Table 5.4.

$$L(\theta) = \hat{E}_t\left[\min\left(\rho_t(\theta)\,\hat{A}_t,\ \ \mathrm{clip}(\rho_t(\theta),\,1-\varepsilon,\,1+\varepsilon)\,\hat{A}_t\right)\right] \tag{5.3}$$

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l\,\delta_{t+l}, \qquad \delta_t = R_t + \gamma V(s_{t+1}) - V(s_t) \tag{5.4}$$

Table 5.4: Training hyperparameters (v7 and v9 protocol)

Parameter	**Value**	**Parameter**	**Value**
Algorithm	PPO (Stable-Baselines3)	Discount γ	0.97
Parallel decisions per step	4 simulations × 20 service agents	GAE λ	0.95
Rollout length	64 steps × 80 agents = 5,120 decisions	Entropy coefficient	0.01
Mini-batch size	512	Value-loss coefficient	0.5
Epochs per update	10	Max gradient norm	0.5
Learning rate (from scratch / after warm start)	3×10^{-4} / 1×10^{-4}	Clip range (from scratch / after warm start)	0.2 / 0.1
Target KL (after warm start)	0.02	Agent-steps per run	2,000,000
Hidden size, message-passing rounds	64, 2	Actor / critic heads	64 / 64 units
Seeds	1 and 2	Validation	every 250,000 steps on 40 fixed workloads

A measured profile showed that the PPO update, not the simulator, dominated training time (77-84% of each iteration on the CPU), so a rule was fixed in advance: train on the

GPU only if it was at least 1.5 times faster. The RTX 3050 was 1.79 times faster than the best CPU setting, and the four v9 runs of 2,000,000 agent-steps each took about 63 minutes in total. The device changed only the speed, never the protocol, seeds or selection.

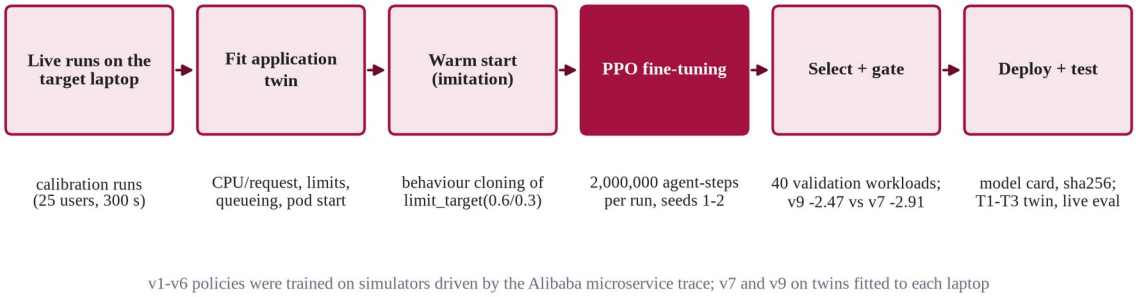


Figure 5.7: Training pipeline of the v7 and v9 policies

5.2.5 Step 5: Selection, gate and model card

Each run was evaluated on 40 fixed validation workloads every 250,000 steps, and the best checkpoint of each run kept. The run with the best validation reward was selected, and it became the deployed policy only if it passed a gate against its predecessor. Figure 5.8 shows the v9 candidates: the selected run `focal_bc_s2` scored −2.466, better than the no-graph variant, the teacher rule and the other seed, and it passed the gate against v7 (−2.47 against −2.91). Because selection and gate used the same validation set, the margin is optimistic; the held-out test sets in Chapter 6 are the fair comparison. The selected model is stored with a model card that records its features, actions, reward, guards, training runs and its SHA-256 fingerprint, and no earlier model was overwritten.

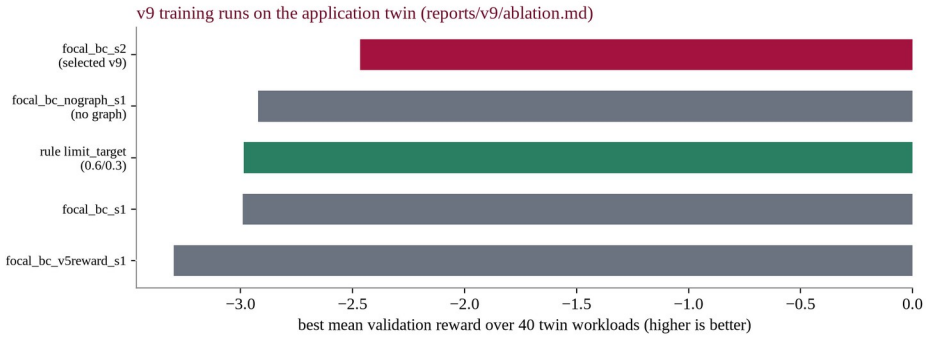


Figure 5.8: Validation reward of the v9 training runs and the teacher rule

5.2.6 Step 6: Live control loop

Algorithm 5.1 gives one operator step. It runs every 10 seconds for the whole application, and Figure 5.9 shows the same loop as a flow.

Algorithm 5.1: FlexaScale operator step (every 10 s)

input: policy file P, trained policy π, Lease L, mode $\in$ {observe, live}, Auto on/off
discover services S and their HPAs (label selector of P)
query Prometheus once for all $s \in S$ (1-minute window); mark missing or stale telemetry
if mode = live **then** renew L; **if** renewal fails **then** restore all HPAs, mode $\leftarrow$ observe
for each $s \in S$ **do** $x(s) \leftarrow$ FeatureTracker.update(s)
$(a(s), c(s)) \leftarrow \pi(x$, call graph, focal = $s)$ for every s
for each service s **do**
    $h(s) \leftarrow \lceil r(s) \cdot \text{CPU}(s) / \text{target} \rceil$ (bounded)
    owner $\leftarrow$ Auto(s, $a(s)$, $c(s)$, $h(s)$) **if** Auto **else** FlexaScale
    if owner = HPA **then** restore the HPA's original bounds; log; **continue**
    $n \leftarrow$ guards($r(s) + a(s)$) (bounds, step, budget, readiness, cooldowns, blocks)
    if mode = live **and** $n \neq r(s)$ **then** freeze HPA at n; scale Deployment to n
    log proposal, approved count, owner, rule, safety reasons, applied flag
write status file (heartbeat for the lab watchdog)

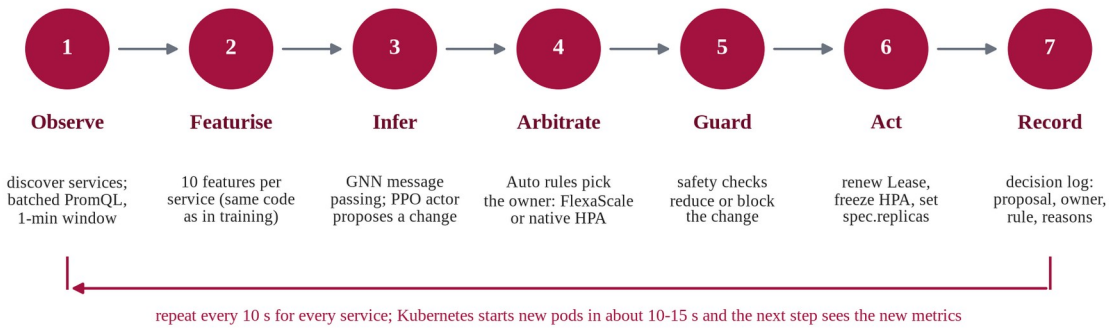


Figure 5.9: The live control loop

5.2.7 Step 7: Auto arbitration

Algorithm 5.2 details the per-service decision of Auto mode. The dwell counter is kept per service across steps.

Algorithm 5.2: Auto arbitration for one service

input: service s, current owner o, policy proposal f, confidence c, HPA recommendation h, telemetry
if no model **or** CPU missing/stale **or** Lease not held **then return** HPA (rule: controller_health)
$\theta \leftarrow$ activation threshold **if** o = HPA **else** deactivation threshold
if $c < \theta$ **then** *candidate* $\leftarrow$ HPA (rule: policy_confidence)
else if (latency > SLO **or** CPU $\geq$ risk level) **and** $h > f$ **then return** HPA (rule: sla_risk)
else *candidate* $\leftarrow$ FlexaScale (rule: efficiency)
if *candidate* = o **then** dwell $\leftarrow$ 0; **return** o
dwell $\leftarrow$ dwell + 1
if dwell ≥ 2 **then** dwell $\leftarrow$ 0; **return** *candidate*
return o (stay with the current owner until the choice is confirmed)

5.2.8 Step 8: Experiment procedure

Every experiment, whether a benchmark pair or a demonstration, is run through the lab so that all controllers are treated identically. Before the load starts, the lab resets the shop's data (restocks inventory, empties carts), scales every service to its minimum, waits until all pods are ready and, if a run has just ended, waits until the HPA's scale-down memory has expired. It then starts the operator in the chosen mode and Locust with the chosen scenario. During the run it samples replicas, metrics, the Locust window and new decisions every five seconds and records events. Afterwards it stops Locust and the operator, restores every HPA, checks that no Lease is held (`cleanup.safe`), and writes the run's files. Section 6.1 describes the scenarios and the rules that make a run valid.

5.2.9 Version history

The final design is the result of nine policy versions. Table 5.5 summarises what each version changed and what was learned from it; the negative results shaped the final method as much as the positive ones.

Table 5.5: Policy versions and what each one taught

Version	Main change	Outcome
v1	First PPO policy on the Alibaba trace	Trained on CPU stored as fractions instead of percent; unit bug found and fixed
v2, v3	Corrected data; v3 adds a cost term (weight 0.5)	Beat an HPA-like rule in simulation only by provisioning about 40% more replicas; live, 58-98% more resources than HPA
v4	Dependency-aware simulator, unit-free inputs, bounded SLO penalty, per-service (focal) objective	52% fewer replicas than v3, still below the HPA rule
v5	Warm start by imitating the HPA-like rule, then small PPO steps	Beat the rule in simulation; live, +8% replica-hours versus HPA
v6	Simulator matching the live control loop; Auto mode	Auto −3.9% replica-hours vs default HPA (18 pairs), later found to be partly a power-profile confound
v7	Application twin fitted to the shop (laptop 1); graph policy over the real call graph	On par with a tuned HPA in a pre-registered, clock-controlled replication (48 runs)
v8	Exploration of the order service's non-CPU bottleneck	Closed: database waits limit the benefit of any replica controller
v9	Twin refitted to laptop 2; GPU training; pre-registered evaluation	−7.3% / −7.7% replica-hours vs default HPA; on par with tuned HPA

Chapter 6

Results and Analysis

6.1 Evaluation setup

The evaluation has two parts with different purposes. The **functional verification** asks whether the system does what it claims on a live cluster: whether Auto mode really hands services to FlexaScale, whether the policy's decisions reach Kubernetes, and whether every failure path returns control to the HPA. The **performance evaluation** asks whether FlexaScale scales better than the native HPA, and it was designed to answer that without bias.

6.1.1 Workloads

Load is generated by Locust with user journeys that browse, search, add to cart and check out, including slow and declined payments. Six scenarios shape the number of users over time (Table 6.1). All benchmark runs used 25 peak users, a spawn rate of 5 users per second and 300 seconds of load; the verification runs used 40 users to create more pressure.

Table 6.1: Workload scenarios

Scenario	Shape of the load
steady_browsing	Constant users, mostly browsing
gradual_ramp	Users grow in six steps to the peak, then plateau
flash_sale	20% of peak for the first 25% of the run, the full peak with checkout contention until 65%, then back to 20%
checkout_heavy	Mostly authenticated cart and checkout journeys at constant load
traffic_drop	Peak for 45% of the run, then 10% (tests scale-down)
payment_degradation	300 ms payment delays and declined cards

6.1.2 Controllers compared

- **Default HPA**: the deployed HPAs at 70% of the CPU request with the default 300 s scale-down window. FlexaScale runs in shadow mode, so the same processes run on the host as in the other arms.
- **Tuned HPA**: a CPU target and a 60 s scale-down window chosen by a calibration rule fixed in advance: among the targets {70, 150, 250, 350}% of request, the highest whose p95, mean latency and failures stay within 10% (0.1 percentage points for failures) of the 70% target. The rule chose 250% on the first laptop and 350% on the second.

- **FlexaScale RL**: the graph policy under the safety layer, owning every service.
- **FlexaScale Auto**: the same policy and guards with the Auto rules; services handed back are scaled by the tuned HPA.

6.1.3 Fairness safeguards

Two pre-registered studies were run, each with its rules written down before the first run: the v7 replication on the first laptop and the v9 evaluation on the second. The v9 study used 12 blocks (six scenarios, each repeated twice). Within a block the four arms ran back to back in an order drawn from a fixed seed, so slow drifts in the host affect all arms alike. Every run started from the same settled state (data reset, every service at one ready replica). A run counted only if it completed at least 95% of its duration, its cleanup was verified safe, the laptop was on AC power in every sample, and the CPU used outside Kubernetes stayed within 2.0 cores of a measured idle baseline. An invalid run was rerun once at the end of its batch. Of 48 runs and 5 reruns, 46 were valid; the seven excluded runs were all FlexaScale or Auto runs affected by host load, and all are kept with the reason for exclusion. An earlier batch whose apparent advantage was traced to the laptop's power profile was withdrawn rather than reported as a result.

6.1.4 Verification setup

The functional verification ran nine flash-sale experiments on the first laptop, each through the normal lab API, and extracted the evidence from the raw decision logs and lab events with a script. Two of them injected faults: the Prometheus port-forward was stopped for 60 seconds from +100 s, and the operator process was killed with SIGKILL at +100 s. A controlled variant of the shop's policy lowered the Auto CPU-risk level to 60% of request and computed the HPA recommendation at 70%, so that the SLA-risk rule would fire under the spike; this variant is used for verification only, never in a benchmark.

6.2 Evaluation criteria

Table 6.2 defines the measures. The primary measure of resource use is replica-hours (Equation 6.1), the integral of ready replicas over the load window; the primary measure of performance is the end-to-end 95th-percentile latency reported by Locust.

$$RH = \frac{1}{3600} \sum_{t} \sum_{i} \mathrm{ready}_i(t)\, \Delta t \tag{6.1}$$

Table 6.2: Evaluation measures

Measure	Definition and unit
Replica-hours	Ready replicas integrated over the load window (Equation 6.1); 16 services at 1 replica for 300 s = 1.333
Estimated cost	Allocated vCPU-hours × $0.04048 + GiB-hours × $0.004445 (AWS Fargate list prices, us-east-1); an estimate, not a bill

Measure	Definition and unit
Latency percentiles	p50, p90, p95, p99 of end-to-end request time (Locust), in ms; per service from Prometheus histograms
SLO violation	Seconds of ~10 s windows in which the end-to-end p95 exceeded 1000 ms
Failure rate	Failed requests ÷ all requests × 100
Throughput	Completed requests per second
Scaling events, oscillations	Changes of desired replicas; reversals within a short time
Paired difference Δ%	100 × (candidate − reference) ÷ reference, computed within a block
95% confidence interval	Mean of the paired differences ± t(0.975, n−1) × s ÷ √n (Equation 6.2)
Effect size d_z	Mean paired difference ÷ standard deviation of the differences

$$\bar{d} \pm t_{0.975,\ n-1} \frac{s_d}{\sqrt{n}} \tag{6.2}$$

The verdict rule was fixed before the runs. A candidate *beats on cost* only if (C1) its replica-hours interval lies entirely below zero and (C2) its p95 is not worse (mean at most +10% and interval not entirely above zero), together with checks on failures (C3), throughput (C4), SLO-violation time (C5), oscillations (C6) and consistency across scenarios (C7). If the intervals include zero the verdict is *on par / inconclusive*.

6.3 Result

6.3.1 End-to-end functional verification

Table 6.3 summarises the nine verification runs. In every run the cleanup was verified safe, and no scale request was ever applied to a service without FlexaScale's ownership and a frozen HPA.

Table 6.3: Controller-mode verification on the live cluster (flash sale, 300 s)

Scenario	Run	Operator steps by owner	Applied scale requests	What was verified
Native HPA, FlexaScale in shadow	001033	HPA 496	0	HPA scaled order and catalog 1→3; 496 proposals logged, none applied
FlexaScale v9	001628	FlexaScale 512	0	Lease taken, every service owned and all decisions accepted; policy held 1 replica
Auto, default policy	002805	FlexaScale 480, HPA 16	1	order-service 1→2 by the policy; desired after 5.9 s, ready after 15.9 s
Auto, SLA-risk variant	035531	FlexaScale 436, HPA 60	7	Mixed ownership in 21 of 31 steps; HPA took risky services
Auto, Prometheus cut 60 s	030452	FlexaScale 336, HPA 128	6	All 16 services to HPA in the same step; FlexaScale retook them and scaled down
Auto, operator SIGKILL	031625	FlexaScale 160, HPA 16	0	Watchdog restored 16 HPAs at +105.2 s; HPA scaled afterwards

Scenario	Run	Operator steps by owner	Applied scale requests	What was verified
FlexaScale v7	032758	FlexaScale 512	0	As v9: owned, accepted, held 1 replica
Auto with v7	033920	FlexaScale 480, HPA 16	0	Efficiency rule throughout; no change proposed
FlexaScale v7, 25 users	040706	FlexaScale 512	0	Held 1 replica at lower load as well

Auto mode selecting FlexaScale. In the default-policy Auto run, the order service was owned by FlexaScale under the efficiency rule when, at +289 s, the v9 policy proposed a scale-up with confidence 0.50. The safety layer accepted it, the operator froze the service's HPA, and Kubernetes reported two desired replicas 5.9 seconds later and two ready replicas 15.9 seconds later. This is the complete chain from metrics to a running pod, recorded in the decision log and confirmed by the Kubernetes API.

Mixed ownership. Table 6.4 gives the per-service evidence of the SLA-risk run, in the format used throughout the verification. The busy services (API gateway, cart, catalogue, inventory and order) moved to the HPA under the SLA-risk rule while the other eleven stayed with FlexaScale in the same step. The HPA scaled the risky services up to three replicas, and FlexaScale applied seven actions: an early scale-up of the order service at +29 s and scale-downs after the spike.

Table 6.4: Per-service evidence, Auto mode with the SLA-risk variant (run 20261031-035531)

Service	Selected owner (steps)	Decision reason (rule: steps)	Policy	Action applied (Kubernetes confirmation)	Replicas desired / ready max
api-gateway	FlexaScale 24, HPA 7	efficiency 26, sla_risk 5	ppo_graph	+249 s 2→1 (desired 1.1 s, ready 1.1 s)	1-2 / 2
cart	FlexaScale 26, HPA 5	efficiency 29, sla_risk 2	ppo_graph	none	1-1 / 1
catalog	FlexaScale 18, HPA 13	efficiency 21, sla_risk 10	ppo_graph	+229 s 3→2; +259 s 2→1	1-3 / 3
inventory	FlexaScale 22, HPA 9	efficiency 26, sla_risk 5	ppo_graph	+239 s 2→1 (6.1 s / 6.1 s)	1-2 / 2
order	FlexaScale 16, HPA 15	efficiency 19, sla_risk 12	ppo_graph	+29 s 1→2 (6.1 s / 16.1 s); +259 s 3→2; +289 s 2→1	1-3 / 3
11 other services	FlexaScale 30, HPA 1 each	efficiency 31	ppo_graph	none	1-1 / 1

Failure handling. When the Prometheus connection was cut at +100 s, every service lost fresh CPU telemetry and the controller-health rule moved all sixteen to the HPA in the same step; their HPAs were unfrozen and the HPA scaled during the outage. After telemetry returned, FlexaScale retook ownership following the two-step dwell and applied six scale-downs. When the operator was killed without warning, the lab watchdog detected

the exited process and restored all sixteen HPAs at +105.2 s, after which the HPA scaled the order service to four replicas and the catalogue to three. Figure 6.1 shows the total ready replicas of four of these runs, and Figure 6.2 the dashboard's replay of the crash run.

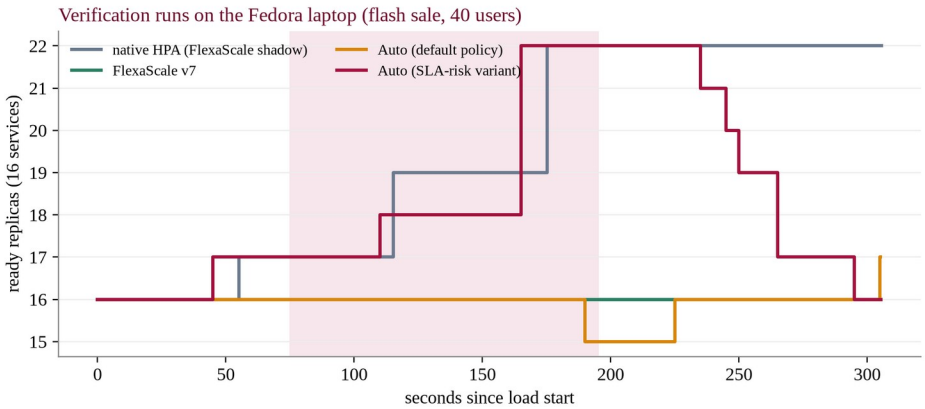


Figure 6.1: Total ready replicas in four verification runs

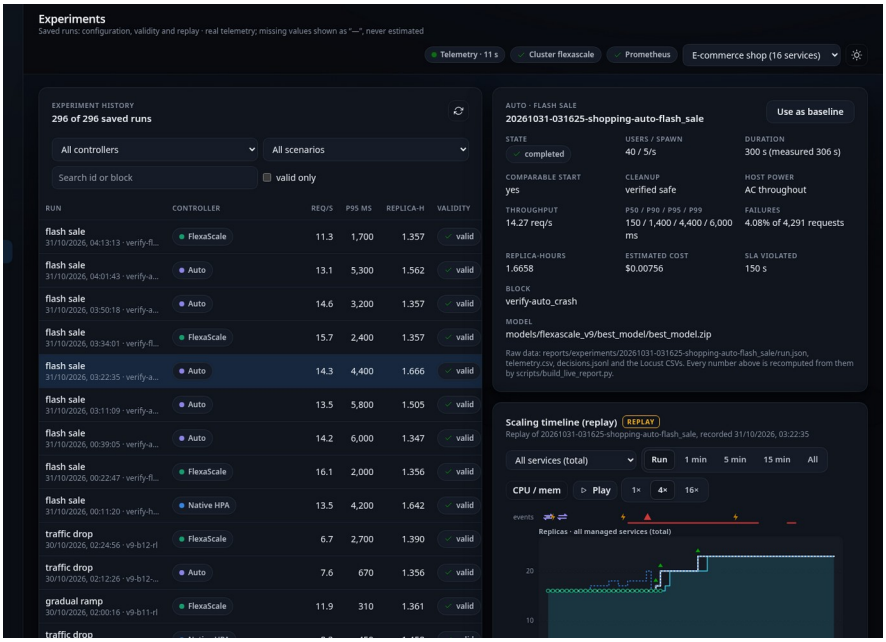


Figure 6.2: Dashboard replay of the operator-kill run: the fallback marker and HPA scaling afterwards

Pure FlexaScale mode. In the three FlexaScale-mode runs on this laptop, ownership, safety and execution worked, but the policies proposed no change: both v7 and v9 held every service at one replica, at 40 users and at 25 users. The reason is examined in Section 6.5. Scale changes in FlexaScale mode had been verified on the same laptop before (run 20261028-061846: order service 1→2 at +115 s, ready at +130 s and back to 1 at +255 s), and in the v9 evaluation on the second laptop (run E2 below).

Other checks. The automated test suite of 373 tests passed. The Helm lifecycle script installed, upgraded, rolled back and uninstalled the chart for the bookstore application and passed all 33 checks; the main release was upgraded from chart 0.4.1 to 0.5.1, rolled back

and upgraded again while the shop's Deployments and the database volume were preserved. For every service, the per-service latency percentiles were computed from the service's own histogram and window, and were ordered p50 ≤ p90 ≤ p95 ≤ p99 in every sample checked.

6.3.2 Results in the application twin

After the live evaluation, every controller was run once on the three held-out twin test sets of the second laptop (Table 6.5). On the six live scenarios (T1), the tuned HPA scored best and v9 was very close. On random variations of the shop's workloads (T2), v9 was best, with the lowest SLO-violation time among the efficient controllers. On random synthetic applications (T3), all controllers struggled, and the learned policies traded fewer replicas for more scaling events.

Table 6.5: Held-out twin test results (episode reward, higher is better; replica-hours; p95 in ms)

Controller	**T1 reward**	**T1 RH**	**T1 p95**	**T2 reward**	**T2 RH**	**T2 p95**	**T3 reward**	**T3 RH**
Default HPA (70%, 300 s)	−6.325	1.477	168	−15.694	2.461	167	−109.3	3.448
Tuned HPA (350%, 60 s)	**−1.196**	1.333	178	−3.060	2.071	217	−82.4	1.673
Rule limit_target(0.6/0.3)	−1.280	1.338	174	−2.789	2.080	208	−79.3	1.802
v9 selected (focal_bc_s2)	−1.255	1.340	171	**−2.548**	2.078	198	−79.0	1.815
v9 no graph (seed 1)	−1.255	1.339	173	−2.839	2.094	194	−76.7	1.969

The graph ablation compared the graph policy with the same network without message passing, trained with the same seed, episode by episode. The difference in reward was −0.001 [−0.026, +0.024] on T1, +0.157 [−0.363, +0.677] on T2 and +0.708 [−3.304, +4.720] on T3. Every interval includes zero, so the graph input showed no measurable benefit on this shop.

6.3.3 Pre-registered live evaluation

Table 6.6 and Figure 6.3 give the mean of every valid run per arm in the v9 evaluation. FlexaScale RL and Auto used the same replica-hours as the tuned HPA and about 10% fewer than the default HPA. Their median latency was the lowest, and their tail latency the highest.

Table 6.6: v9 evaluation: mean over valid runs per arm (second laptop, 25 users, 300 s)

Arm	**Runs**	**Replica-hours**	**Cost (USD)**	**p50 (ms)**	**p95 (ms)**	**p99 (ms)**	**Failures (%)**	**req/s**	**SLO violation (s)**
Default HPA	12	1.512	0.00685	44	241	838	0.29	15.13	22.9
Tuned HPA	12	1.366	0.00618	42	167	510	0.25	15.37	8.0
FlexaScale RL	11	1.368	0.00619	37	395	1674	0.34	15.07	25.6
FlexaScale Auto	11	1.365	0.00618	37	200	1936	0.32	15.15	15.7

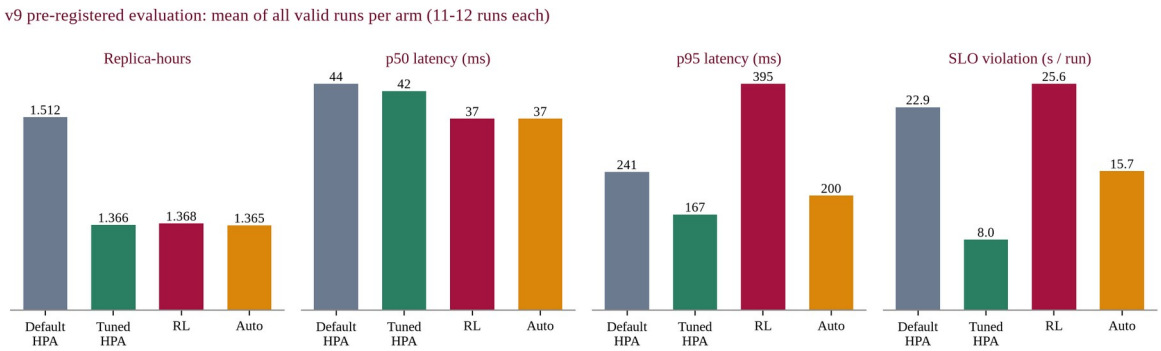


Figure 6.3: v9 evaluation: replica-hours, median and tail latency and SLO-violation time per arm

Table 6.7 gives the paired comparisons with their confidence intervals and the pre-registered verdicts, and Figures 6.4 and 6.5 show them graphically.

Table 6.7: Pre-registered paired comparisons (mean Δ with 95% CI)

Study	Comparison	Pairs	Replica-hours Δ	p95 Δ	Verdict
v9	Tuned HPA vs default HPA	12	**−9.0%** [−13.9, −4.1]	−10.7% [−27.5, +6.1]	Beats on cost
v9	FlexaScale RL vs default HPA	11	**−7.3%** [−11.5, −3.2]	+53.5% [−49.4, +156.4]	On par / inconclusive
v9	FlexaScale Auto vs default HPA	11	**−7.7%** [−12.3, −3.2]	+17.6% [−6.0, +41.2]	On par / inconclusive
v9	FlexaScale RL vs tuned HPA	11	+0.3% [−0.2, +0.7]	+155% [−165, +475]	On par / inconclusive
v9	FlexaScale Auto vs tuned HPA	11	+0.2% [−0.2, +0.5]	+41.5% [−22.6, +105.7]	On par / inconclusive
v7	FlexaScale RL vs tuned HPA	12	+0.1% [−0.1, +0.2]	+7.2% [−8.1, +22.4]	On par / inconclusive
v7	FlexaScale Auto vs tuned HPA	12	+0.1% [−0.1, +0.2]	+0.1% [−9.3, +9.5]	On par / inconclusive
v7	Rule (no model) vs tuned HPA	12	−0.0% [−0.1, +0.0]	+4.5% [−6.6, +15.6]	On par / inconclusive

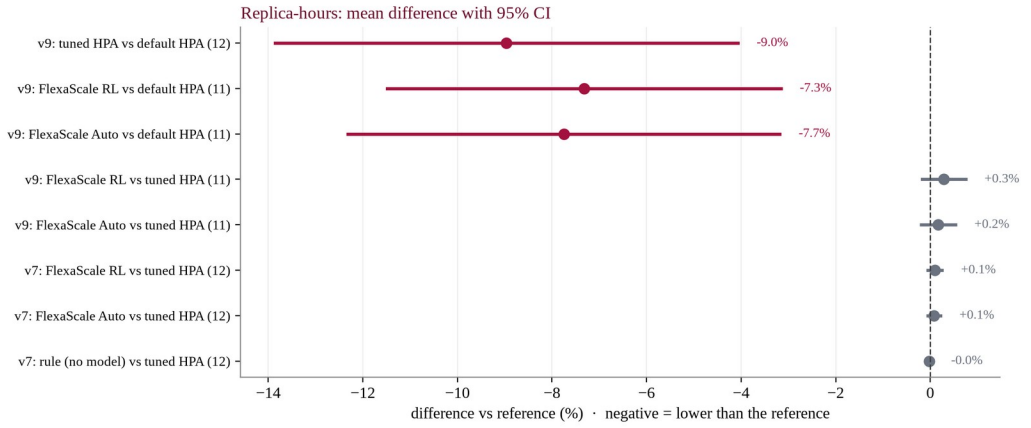


Figure 6.4: Replica-hours: paired differences with 95% confidence intervals

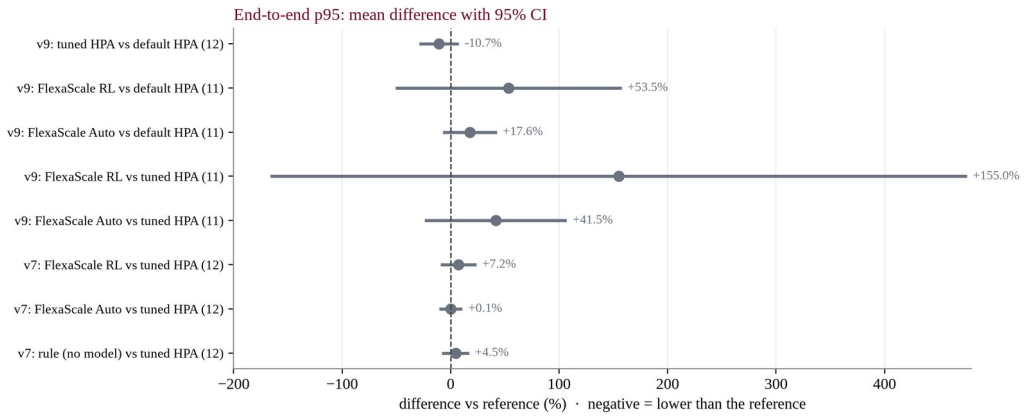


Figure 6.5: End-to-end p95 latency: paired differences with 95% confidence intervals

The p95 means of the FlexaScale arms are strongly affected by three late reruns that ran one to two hours after their blocks while the laptop was short of physical memory. Excluding the pairs that contain them, as a descriptive sensitivity check only, FlexaScale RL against the tuned HPA was +0.1% in replica-hours and +3.8% [−6.2, +13.9] in p95. The verdicts do not change.

6.3.4 Case studies: flash sale and checkout-heavy load

Four blocks of the v9 evaluation show where FlexaScale saves resources. They were selected because a FlexaScale arm did better than the default HPA on a clearly defined measure, so they illustrate the mechanism; the pooled results above, not these cases, are the evidence for or against an overall advantage.

Table 6.8: E1: flash sale, v9 block 4 (Auto) and E2: flash sale, v9 block 10 (RL)

Measure	**E1 default HPA**	**E1 tuned HPA**	**E1 Auto**	**E2 default HPA**	**E2 tuned HPA**	**E2 RL**
Throughput (req/s)	15.62	16.15	**16.95**	13.36	15.24	15.12
p50 / p95 (ms)	130 / 410	120 / 320	130 / **200**	140 / 930	130 / 400	130 / **330**
p99 (ms)	1300	470	340	3200	760	1400
Error rate (%)	0.30	0.43	0.35	0.92	0.37	0.37
Average / peak replicas	22.2 / 27	16.3 / 17	16.6 / 17	21.4 / 28	16.5 / 17	16.6 / 17
Replica-hours	1.829	1.386	**1.410**	1.766	1.398	**1.412**
SLO violation (s)	30	0	**0**	115	5	30.1
Scaling events	8	2	2	10	1	2

In E1 the default HPA reacted to the spike after it had started, reached 27 replicas and kept them for five minutes after the spike ended, and it still spent 30 seconds above the SLO. Auto, with the v9 policy owning the services, added one replica where it was needed and held the rest: 22.9% fewer replica-hours, p95 halved (−51.2%) and no SLO violation (Figure 6.6). E2 repeated the pattern in an independent block with the pure learned policy:

20.1% fewer replica-hours and p95 −64.5%. In both blocks, however, the tuned HPA achieved almost the same replica-hours, with less SLO-violation time in E2.

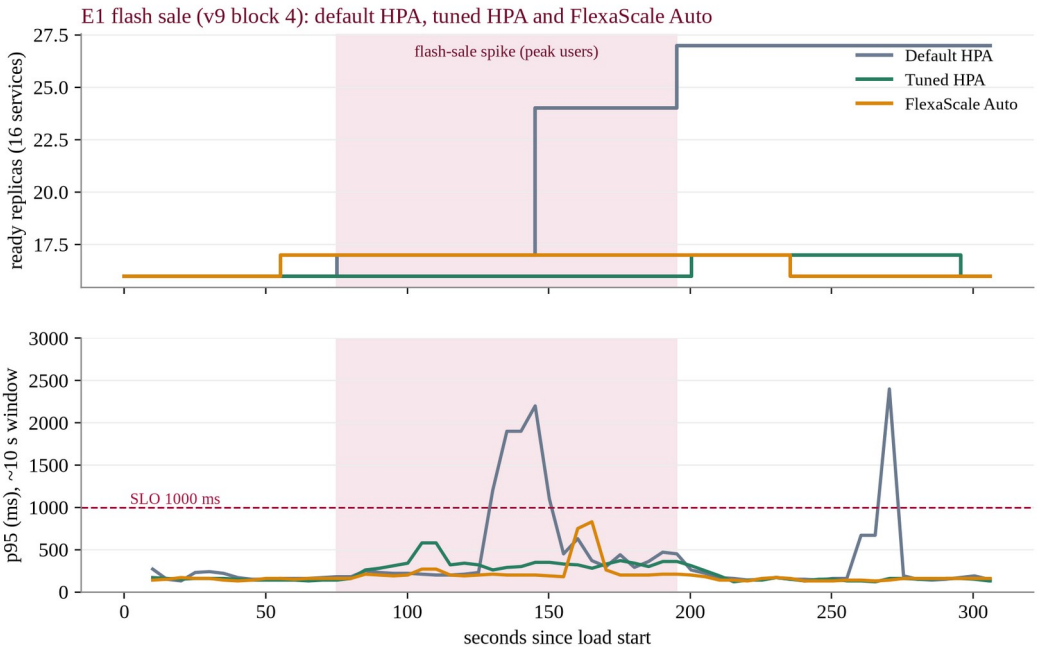


Figure 6.6: E1 flash sale: total ready replicas and end-to-end p95 under default HPA, tuned HPA and Auto

In the checkout-heavy blocks E3 and E4, the default HPA scaled three or four services to two replicas for a CPU signal that did not need them, while FlexaScale (RL and Auto) and the tuned HPA held every service at one replica. FlexaScale used 15.3% and 13.8% fewer replica-hours than the default HPA, at the cost of a p95 about 6% higher (170 against 160 ms, 190 against 180 ms), which is at the resolution of Locust's percentile buckets.

6.3.5 Dashboard outputs

The control center makes every decision visible while it happens and afterwards. Figure 6.7 shows the Live lab during an Auto experiment and Figure 6.8 the Auto mode page after the SLA-risk run, with the owner band, hand-off markers, rule usage (SLA risk 7%, efficiency 93%) and each service's owner and reason.

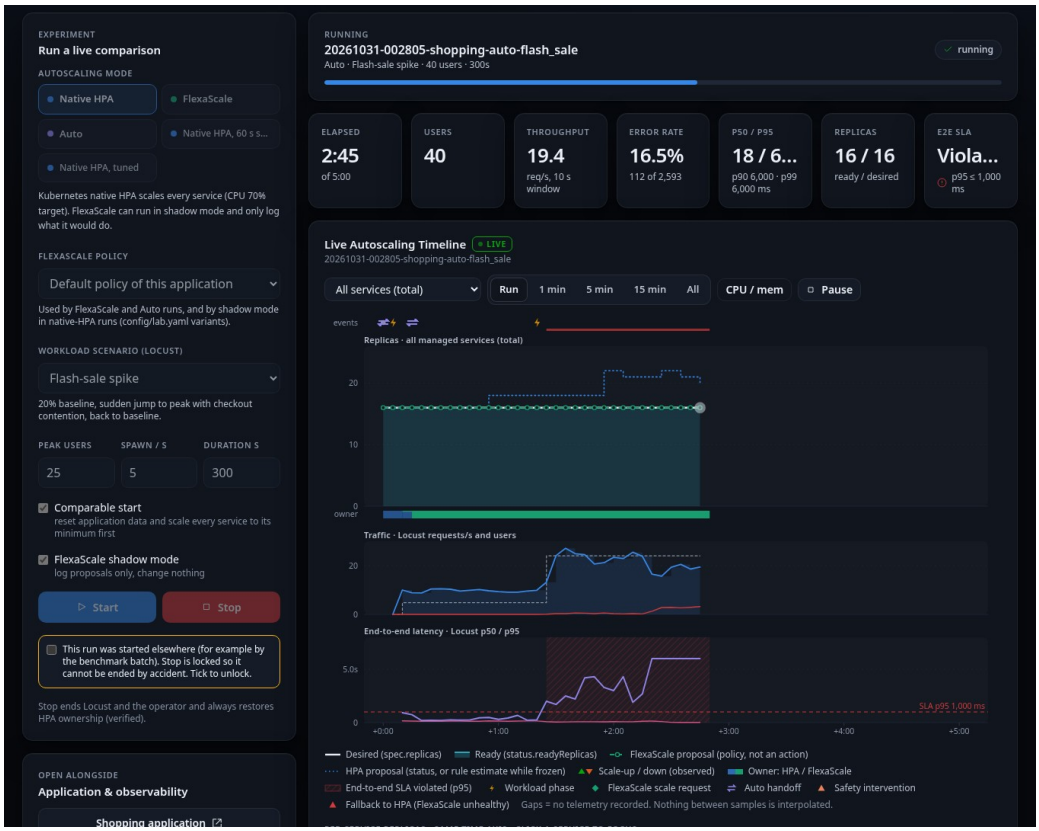


Figure 6.7: Live lab during an Auto-mode flash-sale run: tiles, live autoscaling timeline and per-service replicas

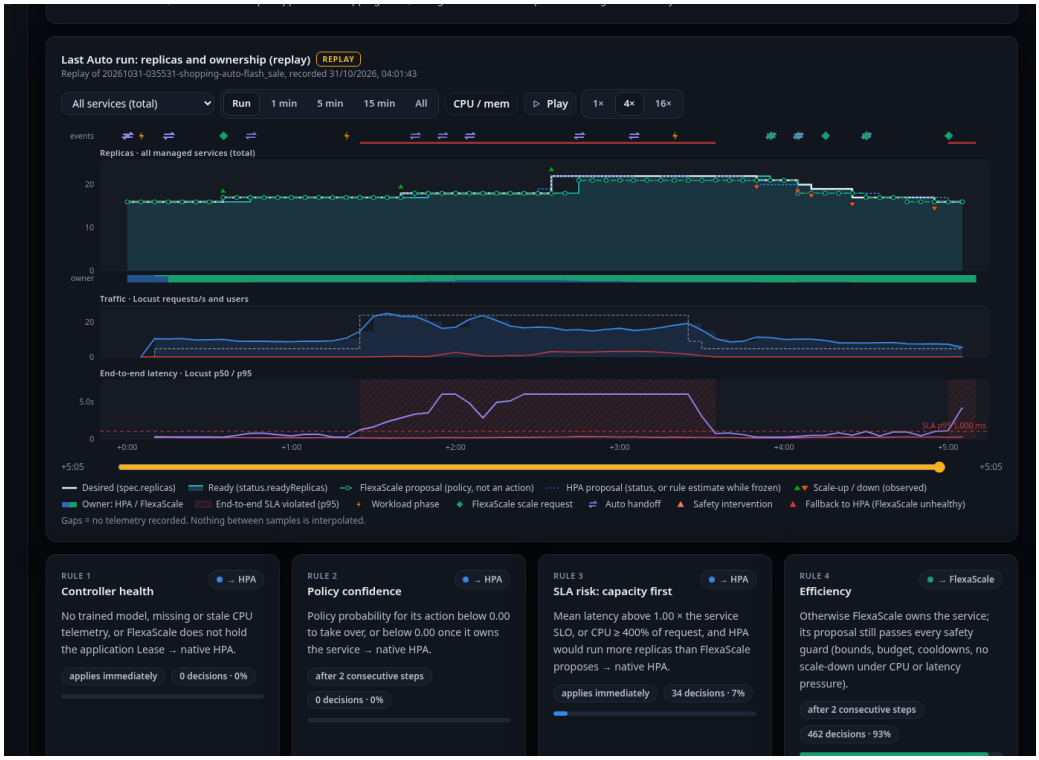


Figure 6.8: Auto mode page: replay of the SLA-risk run and the four rules with their usage

6.4 Comparative Analysis

Against the default HPA. FlexaScale used fewer resources: 7.3% (RL) and 7.7% (Auto) fewer replica-hours with intervals entirely below zero, and an estimated cost lower by 7.4% and 7.9%. Figure 6.9 shows where the saving comes from. The flash-sale and checkout-heavy scenarios carry it, because FlexaScale did not keep spike capacity for the HPA's five-minute window and did not react to brief CPU rises. In gradual ramp and traffic drop, Auto's p95 was 30-65% higher, which is why the p95 check (C2) failed and the pre-registered verdict is *on par / inconclusive* rather than *beats on cost*.

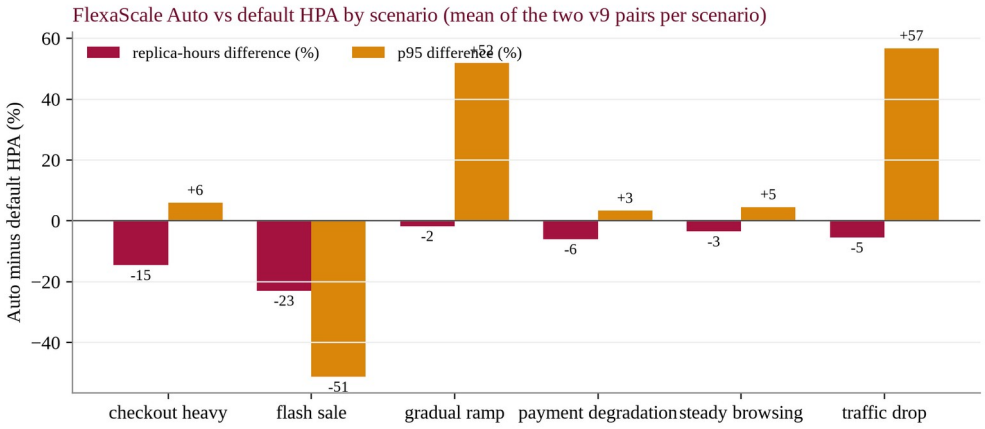


Figure 6.9: FlexaScale Auto minus default HPA, by scenario

Against a tuned HPA. In both pre-registered studies, on two different laptops, FlexaScale was on par: replica-hours within half a percent and p95 intervals that include zero. A tuned HPA achieved the whole saving over the default HPA without any learned model (−9.0% replica-hours). The v7 study also ran a hand-written rule inside the same operator, and it too was on par, which suggests that at these loads the operator's guards and a sensible threshold do most of the work.

Against the literature. The systems reviewed in Chapter 3 report larger gains: for example, FIRM reports up to 16 times fewer SLO violations [7], and Gwydion and Smart HPA report better efficiency than the HPA [1, 2]. These figures are not directly comparable with ours: they come from other applications, other hardware and, in most cases, a comparison with the default HPA only. Against the default HPA, our result points the same way (a 7-8% saving and up to 23% in a flash sale). What this project adds is the comparison with a calibrated HPA under rules fixed in advance, and there the advantage disappears. Table 6.9 places FlexaScale beside the closest related systems on the design questions of Section 2.2.

Table 6.9: FlexaScale compared with the closest related systems

Aspect	**Gwydion [1]**	**FIRM [7]**	**GRAF [10]**	**FlexaScale**
Decision	Replicas (RL)	Resource limits (RL)	CPU allocation (GNN + optimiser)	Replicas (GNN + PPO)

Aspect	Gwydion [1]	FIRM [7]	GRAF [10]	FlexaScale
Dependencies	In the state	Critical-path localisation	GNN latency model	Message passing on the call graph
Coexistence with HPA	Not reported	Not addressed	Not addressed	Per-service Auto arbitration, HPA freeze, Lease
Failure fallback	Not reported	Not reported	Not reported	Verified: telemetry loss, crash, Lease loss
Baseline	Default HPA	Kubernetes autoscaling	Existing autoscalers	Default and tuned HPA
Evaluation rules	Not reported	Not reported	Not reported	Pre-registered, paired, 95% CI

6.5 Discussion and Interpretation

The objectives of Section 1.2 were met as a system: FlexaScale discovers and controls applications it was not written for (O1), runs a trained dependency-aware policy live (O2, O3), keeps the cluster safe and hands control back to the HPA whenever it should (O4), was evaluated fairly against both HPA configurations (O5), and can be watched and operated through its dashboard and installed with Helm (O6). The open question was whether the learned policy is a *better* controller, and the answer from two independent studies is that it is as good as a tuned HPA, not better, on this application and at these loads.

Several findings explain why. At 25 users, most services sit at their minimum of one replica under any sensible controller: the floor of 1.333 replica-hours is within 2-6% of what both the tuned HPA and FlexaScale used, so there was little left to save. A tuned HPA, with a higher target and a short scale-down window, already removes the default HPA's main sources of waste. The graph input did not help measurably, probably because in this shop the load on each service is visible in its own request-rate and CPU features soon enough, and the twin's error in p95 (about 9%) is of the same size as the differences the policy would need to exploit.

The verification runs revealed a more instructive limitation. At 40 users on the first laptop, the order service's latency rose about tenfold while its CPU stayed at 30-50% of its limit, because it waits on synchronous database calls. Both learned policies rely mainly on CPU relative to the limit, so they held one replica. The default HPA, whose target of 70% of the request is only 14% of the limit, added replicas. In Auto mode, the default SLA-risk threshold, calibrated on the second laptop, did not hand these services to the HPA, and the shop's policy had no per-service latency SLO; with the controlled threshold it did. An autoscaler is only as good as the signal it reads and the thresholds calibrated for the

machine it runs on, and making Auto use end-to-end latency directly is the most useful improvement this points to.

The safety design, on the other hand, worked in every case that was tested, including the failures. This matters for the practical goal of the project and for SDG 9: an operator can let FlexaScale manage services it handles well, keep the HPA in charge of the others, and rely on the fallback when anything goes wrong. The resource saving over the default HPA is real and statistically supported, which is the demonstrated contribution to SDG 12, but the same saving is available from tuning the HPA, and that should be said plainly.

6.6 Limitations

- **Scale.** Single-node Minikube on laptops, 25 to 40 users and 300-second runs. Results may differ on multi-node clusters, under heavier load or over longer periods.
- **One main application.** The shop (16 services) is the only application evaluated for performance; the bookstore was used for deployment tests only.
- **Host noise.** Laptop power profiles, memory pressure and background activity affect latency. Validity rules controlled this, and seven runs were excluded, but the variance between repetitions remains large (tens of percent in p95).
- **Per-laptop calibration.** Thresholds and twins fitted on one laptop did not transfer to the other; the v9 policy is the right one for the second laptop and v7 for the first.
- **Signal.** The policies read mainly CPU. Bottlenecks that are not CPU, such as database waits, are seen only indirectly through latency inflation.
- **Statistics.** Eleven or twelve pairs per comparison; case studies E1 to E4 are single pairs and illustrate rather than prove.
- **Cost.** Costs are estimates from list prices applied to allocated resources, not measured bills or energy.

Chapter 7

Conclusion

7.1 Conclusion

This project set out to make Kubernetes autoscaling of microservices more efficient by giving the decision to a learned, dependency-aware policy, without giving up the safety of the native HPA. FlexaScale achieves the second part fully. It is a working framework that discovers an application's services, builds unit-free features from Prometheus telemetry, runs a graph neural network with a PPO policy over the call graph, arbitrates per service between itself and the HPA, bounds every change with a safety layer, and enforces exclusive ownership with a Lease and an HPA freeze. Live verification showed Auto mode selecting FlexaScale and applying scaling that Kubernetes confirmed, mixed ownership under SLA risk, and correct hand-back to the HPA when telemetry was cut or the operator was killed. The system is tested by 373 automated tests, packaged as a Helm chart and observable through a seven-page dashboard.

On the first part, the evidence is mixed and was reported as it came. In a pre-registered evaluation of 46 valid runs, FlexaScale used 7.3-7.7% fewer replica-hours than the default HPA, with up to 23% in a flash sale, but it was on par with a tuned HPA in two independent studies, and the graph input showed no measurable benefit. The project's main contribution is therefore a safe, explainable and installable framework for learned autoscaling, together with an honest measurement of how much a learned policy gains over well-configured rules at small scale: on this application, nothing beyond what tuning provides.

7.2 Future work

The S8 phase will build on these results. The following tasks are planned, each with its dependency and the way it will be validated.

1. **Latency-aware inputs and Auto rules.** Add end-to-end and per-service latency SLOs to the policy's state and to the SLA-risk rule, so that non-CPU bottlenecks are handled. Validation: the 40-user verification scenario, where the current policies held one replica.
2. **Automatic per-cluster calibration.** Derive Auto thresholds and the tuned-HPA baseline from a short calibration run on each new cluster instead of carrying them across machines. Validation: repeat the v9 protocol on both laptops.
3. **Stronger baselines.** Compare with per-service tuned HPAs, custom-metric HPAs and a faster metrics resolution, which could close the gaps FlexaScale shows.

4. **Larger clusters and loads.** Run on a multi-node cluster with heavier and longer workloads, where more services leave their minimum and dependency information has more room to help. This depends on access to cloud credits or a lab cluster.
5. **Safe online adaptation.** Fine-tune the policy from live data in shadow mode only, and deploy a new checkpoint after it passes the same gate and held-out tests.
6. **Publication and dissertation.** Prepare a paper on fair evaluation of learned autoscalers against tuned baselines for one of the venues identified in the initial submission, and complete the final dissertation and demonstration.

References

[1] J. Santos, E. Reppas, T. Wauters, B. Volckaert, and F. De Turck, “Gwydion: Efficient auto-scaling for complex containerized applications in Kubernetes through reinforcement learning,” *J. Netw. Comput. Appl.*, vol. 234, Art. no. 104067, 2025, doi: 10.1016/j.jnca.2024.104067.

[2] H. Ahmad, C. Treude, M. Wagner, and C. Szabo, “Towards resource-efficient reactive and proactive auto-scaling for microservice architectures,” *J. Syst. Softw.*, vol. 225, Art. no. 112390, 2025, doi: 10.1016/j.jss.2025.112390.

[3] V. Pandey, “Reinforcement learning-based autoscaling for cost and performance optimization in Kubernetes clusters,” 2025.

[4] Y. Shi, M. Dong, W. Zhang, L. Liu, Y. Zheng, L. Cui, and J. Zhang, “AdaptScale: An adaptive data scaling controller for improving multiple performance requirements in clouds,” *Future Gener. Comput. Syst.*, vol. 105, pp. 814–823, 2020, doi: 10.1016/j.future.2017.08.034.

[5] G. El Haj Ahmed, F. Gil-Castiñeira, and E. Costa-Montenegro, “KubCG: A dynamic Kubernetes scheduler for heterogeneous clusters,” *Softw. Pract. Exp.*, vol. 51, no. 2, pp. 213–234, 2021, doi: 10.1002/spe.2898.

[6] F. Rossi, M. Nardelli, and V. Cardellini, “Horizontal and vertical scaling of container-based applications using reinforcement learning,” in *Proc. IEEE 12th Int. Conf. Cloud Computing (CLOUD)*, Milan, Italy, 2019, pp. 329–338, doi: 10.1109/CLOUD.2019.00061.

[7] H. Qiu, S. S. Banerjee, S. Jha, Z. T. Kalbarczyk, and R. K. Iyer, “FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices,” in *Proc. 14th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2020, pp. 805–825.

[8] K. Rzadca *et al.*, “Autopilot: Workload autoscaling at Google,” in *Proc. 15th European Conf. Computer Systems (EuroSys)*, Heraklion, Greece, 2020, Art. no. 16, doi: 10.1145/3342195.3387524.

[9] Y. Zhang, W. Hua, Z. Zhou, G. E. Suh, and C. Delimitrou, “Sinan: ML-based and QoS-aware resource management for cloud microservices,” in *Proc. 26th ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021, pp. 167–181, doi: 10.1145/3445814.3446693.

[10] J. Park, B. Choi, C. Lee, and D. Han, “GRAF: A graph neural network based proactive resource allocation framework for SLO-oriented microservices,” in *Proc. 17th Int. Conf. emerging Networking EXperiments and Technologies (CoNEXT)*, 2021, pp. 154–167, doi: 10.1145/3485983.3494866.

[11] S. Luo *et al.*, “Characterizing microservice dependency and performance: Alibaba trace analysis,” in *Proc. ACM Symp. Cloud Computing (SoCC)*, Seattle, WA, USA, 2021, pp. 412–426, doi: 10.1145/3472883.3487003.

[12] Y. Gan *et al.*, “An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems,” in *Proc. 24th Int. Conf. Architectural*

Support for Programming Languages and Operating Systems (ASPLOS), Providence, RI, USA, 2019, pp. 3–18, doi: 10.1145/3297858.3304013.

[13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.

[14] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016, arXiv:1506.02438.

[15] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

[16] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017, arXiv:1609.02907.

[17] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, Sydney, Australia, 2017, pp. 1263–1272.

[18] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *Commun. ACM*, vol. 59, no. 5, pp. 50–57, 2016, doi: 10.1145/2890784.

[19] The Kubernetes Authors, “Horizontal Pod Autoscaling,” Kubernetes Documentation. [Online]. Available: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/ (accessed Oct. 2026).

[20] Alibaba Group, “Alibaba cluster trace program: cluster-trace-microservices-v2021.” [Online]. Available: https://github.com/alibaba/clusterdata (accessed Oct. 2026).

[21] United Nations General Assembly, “Transforming our world: The 2030 Agenda for Sustainable Development,” Resolution A/RES/70/1, 2015.

Appendix A

Source Code and Reproducibility

The complete source code, configuration, trained models, raw experiment data and analysis scripts are in the project repository below (branch `main`, commit `d8416a2`). No passwords, tokens or private keys are stored in it; the shop's demo user account exists only inside the local cluster. The Alibaba raw trace (7.4 GB) is not in the repository and must be downloaded from its public source; the processed dataset and its provenance file are included.

```
https://github.com/ashwinpraveengo/flexascale
```

A.1 Setup and execution

Prerequisites: Docker, Minikube, kubectl, Helm 3, Python 3.10 or later and Node.js (only to rebuild the dashboard). **Installation:**

```
git clone https://github.com/ashwinpraveengo/flexascale.git
cd flexascale
python -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt && pip install -e .
./fsctl start    # cluster, monitoring, shop, checks, dashboard
                 # dashboard at http://127.0.0.1:8090/
./fsctl status   # owner of scaling, Lease, model, processes
./fsctl stop     # stop all, restore every HPA, verify no Lease
```

Running an experiment. In the dashboard's Live lab, choose a mode (Native HPA, FlexaScale or Auto), a policy, a scenario and the number of users, and press Start. On the first laptop choose the v7 policy; the default v9 policy was fitted to the second laptop. Results appear under Experiments and Comparison and are written to `reports/experiments/<run id>/`.

Reproducing the results and checks:

```
pytest -q                                    # 373 tests
python scripts/build_results.py             # results/ from raw runs
python scripts/verify_controller_modes.py --report
bash scripts/helm_lifecycle_test.sh          # Helm lifecycle checks
python scripts/train_v7.py --help            # v7/v9 training protocol
```

A.2 Supporting material

Table A.1 maps the repository to the parts of this report, and Table A.2 lists the main groups of automated tests.

Table A.1: Repository map

Path	Content
src/flexascale/operator/	Operator (app_operator.py) and Auto rules (auto_mode.py)
src/flexascale/rl/	Features (features_v3.py), graph policy (graph_policy.py), runtime, imitation
src/flexascale/simulator/	Alibaba-driven environments (v1-v6) and the application twin (v7, v9)
src/flexascale/safety/	HPA freeze and restore, Lease, ownership locks
src/flexascale/metrics/, discovery/	Prometheus telemetry and service discovery
src/flexascale/lab/	Lab server, experiment runner, benchmark and comparison
dashboard/	React and TypeScript control center
apps/, charts/shopping-app/	The 16-service shop and its Helm chart
helm/flexascale/	FlexaScale Helm chart 0.5.1
config/apps/, config/twin/	Application policies and fitted twins
models/	Trained policies v2-v9 with model cards
reports/, results/	Raw runs, pre-registered analyses, verification, charts and screenshots
docs/	Pre-registrations (POLICY_V4-V9), runbook, learning guides

Table A.2: Main groups of automated tests (373 tests in total, all passing)

Test file	What it checks
test_auto_mode.py	Every Auto rule, the order of rules and the two-step dwell
test_operator.py, test_agnostic_operator.py	Guards, budget, Lease loss, application-agnostic discovery
test_safety.py	HPA freeze and restore from annotations
test_workload_metrics.py	Pod-level CPU fallback, double scraping, zero versus missing, p90 and capped percentiles
test_lab.py	Process safety, lab API, operator watchdog, p90 backfill
test_helm_chart.py	Chart rendering, release-specific selector, hand-back hook
test_hpa_tuned.py	Tuned-HPA arm and model tags across operating systems